



Módulo Ingressos



AquaVista — Notion Projeto Mestre V2

Versão otimizada para importação no Notion com cobertura ampliada de arquivos, bancos de dados e templates operacionais

Resposta objetiva

Não. O plano consolidado da Versão 4 cobre muito bem a arquitetura, os módulos, os tipos de dados, as rotas, os componentes obrigatórios, os serviços, os seeds principais, QA, Playwright e o complemento de mapas/destinos, mas ele não lista de forma exaustiva todos os arquivos individuais de todas as pastas do projeto.

Esta Versão 2 foi preparada para resolver essa lacuna. Além da base funcional da Versão 4, ela acrescenta uma camada de importação pensada para o Notion com manifesto de arquivos, bancos de dados sugeridos, propriedades, templates de páginas, backlog, dependências, critérios de aceite e organização pronta para transformar o documento em projeto executável.

O que a Versão 4 já cobre

- Prompt mestre de execução do produto.
- Prompts por fase do projeto.
- Estrutura macro de pastas: `app`, `components`, `lib`, `services`, `store`, `mocks`, `tests` e `public`.
- Seeds principais: `parks.json`, `categories.json`, `products.json`, `availability.json`, `coupons.json`, `users.json`, `faq.json` e `policy.json`.
- Módulo adicional de destinos, mapas, rotas, Google Maps e catálogo editorial.
- Seed adicional `park-destinations.json`.

- Critérios de QA, Playwright e nota final de implementação.

O que esta Versão 2 acrescenta

- Manifesto de arquivos recomendado, mais detalhado e pronto para virar tarefas no Notion.
- Estrutura de databases do Notion com propriedades sugeridas.
- Divisão por épicos, fases, milestones e entregáveis.
- Templates de páginas para backlog, bugs, assets, seeds e decisões técnicas.
- Mapeamento mais explícito entre pasta, arquivo, responsabilidade e fase.
- Formato mais amigável para o Notion AI gerar board, timeline e database.

Estrutura de pastas e arquivos — manifesto ampliado

A estrutura abaixo serve como base para o Notion criar tarefas por arquivo, por módulo ou por fase.

App e rotas

- `app` — layout principal, rotas públicas, checkout, conta e APIs mock.
- `app/(public)/page.tsx` — landing institucional.
- `app/(public)/tickets/page.tsx` — hub de ingressos.
- `app/(public)/water-park/p/page.tsx` — página principal de compra.
- `app/(public)/faq/page.tsx` — FAQ.
- `app/(public)/purchase-policy/page.tsx` — política comercial.
- `app/(public)/destinations/page.tsx` — listagem de destinos.
- `app/(public)/destinations/[slug]/page.tsx` — página individual do destino.
- `app/(public)/destinations/category/[category]/page.tsx` — categoria editorial.
- `app/(public)/destinations/city/[city]/page.tsx` — destinos por cidade.
- `app/(public)/destinations/map/page.tsx` — mapa geral.
- `app/(public)/route/page.tsx` — rota parametrizada.

- `app/(public)/route/preview/[slug]/page.tsx` — preview de rota.
- `app/(checkout)/cart/page.tsx` — carrinho.
- `app/(checkout)/checkout/identify/page.tsx` — etapa de identificação.
- `app/(checkout)/checkout/customer/page.tsx` — dados do cliente.
- `app/(checkout)/checkout/payment/page.tsx` — pagamento.
- `app/(checkout)/checkout/review/page.tsx` — revisão.
- `app/(checkout)/checkout/success/page.tsx` — sucesso.
- `app/(account)/account/login/page.tsx` — login.
- `app/(account)/account/register/page.tsx` — cadastro.
- `app/(account)/account/orders/page.tsx` — lista de pedidos.
- `app/(account)/account/orders/[id]/page.tsx` — detalhe do pedido.

APIs mock

- `app/api/mock/catalog/route.ts` — catálogo mock.
- `app/api/mock/availability/route.ts` — disponibilidade mock.
- `app/api/mock/pricing/route.ts` — pricing mock.
- `app/api/mock/checkout/route.ts` — checkout mock.
- `app/api/mock/orders/route.ts` — pedidos mock.

Componentes

- `components/layout/*` — header, footer, navegação, menu mobile e faixas promocionais.
- `components/calendar/*` — calendário, legenda, tooltip, controles e painéis de datas.
- `components/catalog/*` — cards, listas, badges, preços, detalhes, seletores de combo e passaporte.
- `components/cart/*` — drawer, resumo, linhas do carrinho, cupom e estado vazio.
- `components/checkout/*` — stepper, formulários, revisão, aceite e sucesso.
- `components/account/*` — login, cadastro, pedidos e detalhes.

- `components/content/*` — FAQ, policy e blocos editoriais de destino.
- `components/maps/*` — mapa, planner de rota, resumo de rota e abrir no Google.
- `components/feedback/*` — loading, empty, error, inline notice e blocking banner.

Bibliotecas e utilitários

- `lib/types/domain.ts` — tipos centrais do domínio.
- `lib/constants/routes.ts` — constantes de rotas.
- `lib/utils/*` — helpers diversos.
- `lib/validators/*` — schemas Zod.
- `lib/formatters/*` — moeda, data e labels.

Serviços

- `services/catalog/*` — serviços de catálogo.
- `services/availability/*` — serviços de disponibilidade.
- `services/pricing/*` — serviços de preço.
- `services/auth/*` — sessão e autenticação mock.
- `services/cart/*` — validação e normalização do carrinho.
- `services/checkout/*` — validação e criação de pedido.
- `services/orders/*` — consulta de pedidos.
- `services/policy/*` — FAQ e política.
- `services/maps/*` — provider, rota, geolocalização e links Google.

Stores

- `store/use-catalog-store.ts` — estado do catálogo.
- `store/use-cart-store.ts` — estado do carrinho.
- `store/use-checkout-store.ts` — estado do checkout.
- `store/use-session-store.ts` — estado da sessão.

Seeds e mocks

- `mocks/data/parks.json` — seed parques.
- `mocks/data/categories.json` — seed categorias.
- `mocks/data/products.json` — seed produtos.
- `mocks/data/availability.json` — seed disponibilidade.
- `mocks/data/coupons.json` — seed cupons.
- `mocks/data/users.json` — seed usuários.
- `mocks/data/faq.json` — seed FAQ.
- `mocks/data/policy.json` — seed política.
- `mocks/data/park-destinations.json` — seed destinos.

Testes e configuração

- `tests/unit/*` — testes unitários.
- `tests/integration/*` — testes de integração.
- `tests/e2e/*` — testes Playwright.
- `README.md` — documentação de uso.
- `.env.example` — variáveis de ambiente de exemplo.
- `package.json` — dependências.
- `playwright.config.ts` — config Playwright.
- `vitest.config.ts` — config Vitest.
- `tailwind.config.ts` — config Tailwind.
- `tsconfig.json` — config TypeScript.
- `eslint.config.*` — lint.
- `prettier.config.*` — formatação.

Databases recomendados no Notion

1. DB 1 — Épicos

- Nome
- Objetivo
- Status

- Owner
- Prioridade
- Módulo
- Fase
- Dependências
- Critério de aceite
- Links relacionados

2. DB 2 — Tarefas

- Título
- Tipo
- Status
- Sprint/Fase
- Pasta
- Arquivo
- Módulo
- Complexidade
- Prioridade
- Responsável
- Data alvo
- Critério de aceite
- Testes relacionados

3. DB 3 — Rotas e Páginas

- Rota
- Tipo de página
- Módulo
- Componentes usados
- Store usada
- Serviço usado

- QA
- Playwright
- Status

4. DB 4 — Componentes

- Componente
- Pasta
- Propósito
- Dependências
- Props-chave
- Módulo
- Estado
- Teste
- Observações

5. DB 5 — Serviços e Stores

- Nome
- Tipo
- Responsabilidade
- Inputs
- Outputs
- Arquivo
- Módulo
- Dependências
- Testes

6. DB 6 — Seeds e Mocks

- Arquivo
- Domínio
- Quantidade de registros
- Fonte interna

- Status de revisão
- Última atualização

7. DB 7 — Bugs e Ajustes

- Título
- Severidade
- Ambiente
- Módulo
- Passos
- Resultado esperado
- Resultado atual
- Evidência
- Status

8. DB 8 — QA e Testes

- Caso
- Tipo
- Suite
- Cobertura
- Pré-condição
- Resultado esperado
- Status

9. DB 9 — Conteúdo e Mídia

- Asset
- Tipo
- Destino
- Categoria editorial
- Origem
- Status
- Prioridade

- Uso no front

10. **DB 10 — Decisões Técnicas**

- Título
- Contexto
- Decisão
- Impacto
- Alternativas consideradas
- Responsável
- Data

Templates de páginas no Notion

Template de Épico

- Objetivo
- Escopo
- Arquivos impactados
- Dependências
- Riscos
- Critérios de aceite
- Checklist de QA
- Links para tarefas filhas

Template de Tarefa Técnica

- Resumo
- Arquivo/pasta
- Entrada
- Saída esperada
- Passo a passo
- Critério de aceite

- Testes necessários
- Observações

Template de Página de Rota

- Rota
- Descrição
- Componentes
- Store
- Serviços
- Estados visuais
- Eventos
- QA

Template de Bug

- Ambiente
- Passos para reproduzir
- Resultado atual
- Resultado esperado
- Severidade
- Evidência
- Correção sugerida

Template de Seed/Mock

- Arquivo
- Schema
- Exemplo
- Regra de atualização
- Relações com outros arquivos
- Status de revisão

Template de Decisão Técnica

- Problema
- Opções
- Decisão final
- Motivo
- Impactos no código
- Tarefas derivadas

Backlog inicial pronto para o Notion

- Épico 1 — Fundação do projeto e configuração da stack.
- Épico 2 — Design system e layout global.
- Épico 3 — Página de compra `/water-park/p`.
- Épico 4 — Carrinho e minicart.
- Épico 5 — Checkout em 5 etapas.
- Épico 6 — Conta do cliente e pedidos.
- Épico 7 — FAQ e política comercial.
- Épico 8 — Catálogo de destinos e conteúdo editorial.
- Épico 9 — Mapas, rotas internas e Google Maps.
- Épico 10 — QA, testes automatizados e README.

Mapeamento de cobertura

Área	Versão 4	Versão 2 Notion
Arquitetura funcional	Coberta	Coberta
Rotas principais	Cobertas	Cobertas + database de rotas
Componentes principais	Cobertos	Cobertos + organização por pasta/arquivo
Seeds principais	Cobertos	Cobertos + governança de seed
Mapas e destinos	Cobertos	Cobertos + database editorial/mídia
Manifesto completo de arquivos	Parcial	Ampliado

Área	Versão 4	Versão 2 Notion
Templates para Notion	Não	Sim
Properties para databases	Não	Sim
Estrutura pronta para board/timeline	Parcial	Sim

Prompt curto para colar no Notion AI



Monte um workspace de projeto para o sistema AquaVista com databases separados para Épicos, Tarefas, Rotas e Páginas, Componentes, Serviços e Stores, Seeds e Mocks, QA e Testes, Bugs e Ajustes, Conteúdo e Mídia e Decisões Técnicas. Use a estrutura de arquivos e o manifesto deste documento para criar itens automaticamente. Organize o cronograma em 10 épicos, relacione cada tarefa ao arquivo/pasta correspondente, e crie visualizações de Board por Status, Timeline por Fase, Lista por Módulo e Tabela por Pasta.

Plano Consolidado V4 — Base Executiva e Operacional

AquaVista Tickets — Plano Consolidado Versão 4

Prompt mestre, fases, QA, Playwright, seeds, mapas, rotas e catálogo de destinos.

Objetivo: documento único para execução técnica, validação e apresentação do projeto AquaVista Tickets.

Escopo: ticketing, catálogo editorial, módulo de mapas, rotas, destinos e critérios de qualidade.

Formato: documento consolidado em uma única versão executiva e operacional.

Status: pronto para implementação, validação interna e repasse para squads/provedores.

Estrutura do documento consolidado

1. Visão executiva e arquitetura geral.

2. Prompt mestre.
3. Prompts por fase.
4. Checklist de QA.
5. Critérios de Playwright.
6. Seed JSON inicial.
7. Complemento de mapas, rotas e catálogo de destinos.
8. Seed adicional de destinos.

Visão executiva e arquitetura geral

Esta versão consolidada reúne a base completa do projeto AquaVista Tickets em um único documento operacional. O objetivo é servir simultaneamente como artefato de execução técnica, referência de produto, checklist de qualidade e pacote de repasse para implementação.

A solução cobre dois blocos integrados:

- o funil de ingressos com catálogo, carrinho e checkout;
- o módulo editorial-operacional de destinos, mapas e rotas, capaz de sustentar descoberta, comparação, planejamento de chegada e pós-venda no mesmo ecossistema.

Princípios do projeto

- Implementação original, sem copiar marca, identidade, ativos ou estrutura literal de terceiros.
- Equivalência funcional robusta para ticketing, conteúdo editorial e navegação até o destino.
- Arquitetura modular, preparada para mocks, testes, futura integração com APIs reais e automação ponta a ponta.
- Camada própria de mídia, conteúdo e confiabilidade interna, evitando redirecionar o usuário para canais externos dos parques.

Estrutura macro da plataforma

- **Módulo de ingressos:** catálogo, pricing, disponibilidade, carrinho, checkout e pedidos.

- **Módulo de destinos:** parques, clubes aquáticos, balneários, perfis editoriais e comparações.
- **Módulo de mapas e rotas:** mapa interno, "como chegar", rotas internas e abertura opcional no Google Maps.
- **Camada de QA:** critérios funcionais, testes automatizados, cenários Playwright e governança de dados mock.

Prompt mestre

Use este bloco como instrução base para a implementação completa do projeto.



Você é um engenheiro principal full-stack responsável por construir uma aplicação web original de venda de ingressos para parque aquático, com arquitetura moderna, regras de negócio realistas, UX premium e navegação completa.

Diretrizes centrais

- Projeto original, sem copiar marca, textos, layout, HTML, CSS, assets ou identidade visual de terceiros.
- Solução própria com equivalência funcional e profundidade comercial.
- Produto com seleção de data, preços dinâmicos, produtos simples, passaportes, combos, ingressos especiais, cabanas, minicart, carrinho, checkout multi-etapas, login/cadastro, conta do cliente, FAQ e política comercial.
- Marca fictícia: **AquaVista Tickets, AquaVista Water Park, Bosque Aventura e AquaVista Cabanas.**

Stack base da Versão 4 original

- Next.js 14+ App Router
- React 18+
- TypeScript strict
- Tailwind CSS
- shadcn/ui
- Zustand
- TanStack Query
- React Hook Form
- Zod
- date-fns
- Framer Motion
- Lucide React
- Sonner
- MSW ou mock service interno

- Vitest
- Testing Library
- Playwright
- ESLint
- Prettier

Arquitetura

- app routes
- components reutilizáveis
- services
- stores
- mocks
- validators
- tests
- README

Rotas base

- `/`
- `/tickets`
- `/water-park/p`
- `/water-park/p?date=YYYY-MM-DD`
- `/cart`
- `/checkout/identify`
- `/checkout/customer`
- `/checkout/payment`
- `/checkout/review`
- `/checkout/success`
- `/account/login`
- `/account/register`
- `/account/orders`

- `/account/orders/[id]`
- `/faq`
- `/purchase-policy`

Regras de negócio obrigatórias

- data obrigatória para cálculo de preço final;
- datas passadas bloqueadas;
- datas esgotadas indisponíveis;
- preço por data;
- Pix preferencial quando aplicável;
- compra para o mesmo dia restringe pagamento a Pix;
- passaporte 2 dias exige 2 datas;
- passaporte 10 dias exige 10 datas;
- combo exige datas dos dois parques;
- cabana exige ingresso principal;
- carrinho vazio não avança;
- checkout exige aceite da política;
- cupom válido recalcula total;
- cupom inválido mostra erro;
- finalização gera pedido mock.

Prompts por fase

Cada fase pode ser executada separadamente para reduzir ambiguidade e permitir acompanhamento incremental.

Fase 1 — Estrutura, tipos, mocks e base do projeto

Objetivo da fase: criar a base do projeto com stack, estrutura, tipos, mocks e serviços iniciais.

Entregas principais

- inicialização do projeto com a stack da Versão 4;
- estrutura de pastas `app`, `components`, `lib`, `services`, `store`, `mocks`, `tests` e `public`;
- tipos de domínio como `Park`, `VisitDate`, `TicketProduct`, `CartItem`, `Customer`, `Order`, `Coupon`, `PaymentMethod` e `CheckoutStep`;
- seed inicial com `parks.json`, `categories.json`, `products.json`, `availability.json`, `coupons.json`, `users.json`, `faq.json` e `policy.json`;
- serviços base de catálogo, disponibilidade, pricing, autenticação, carrinho, checkout, pedidos e política;
- stores Zustand para catálogo, carrinho, checkout e sessão.

Resultado esperado

- projeto compila;
- estrutura pronta;
- mocks carregáveis;
- types consistentes;
- serviços e stores iniciais criados.

Fase 2 — Layout global e páginas públicas

Objetivo da fase: construir layout global e páginas públicas iniciais com identidade original.

Entregas principais

- layout global com header, promo bar, navegação principal, menu mobile e footer;
- design system base com tokens, spacing, sombras e tipografia;
- páginas `/`, `/tickets`, `/faq` e `/purchase-policy`;
- estados globais de loading, empty, error e toast.

Resultado esperado

- páginas públicas navegáveis;
- visual premium e original;
- conteúdo institucional próprio.

Fase 3 — Página principal do parque, calendário, catálogo, preços e minicart

Objetivo da fase: construir a experiência principal de compra na rota `/water-park/p`.

Entregas principais

- calendário com dois meses e query param `?date=YYYY-MM-DD`;
- componentes de calendário, cards de produto, seletores de datas, alerts e minicart;
- regras de bloqueio de datas passadas e sold out;
- atualização visual de preços ao selecionar data;
- seções obrigatórias: ingressos 1 dia, passaportes, combos, especiais e cabanas;
- validação de add to cart para todos os tipos de produto.

Resultado esperado

- rota principal funcional e convincente;
- calendário integrado ao catálogo;
- minicart funcionando;
- regras visíveis refletidas no front.

Fase 4 — Carrinho e checkout completo

Objetivo da fase: construir carrinho e checkout completo em 5 etapas.

Entregas principais

- rotas `/cart`, `/checkout/identify`, `/checkout/customer`, `/checkout/payment`, `/checkout/review` e `/checkout/success`;
- componentes de linha de carrinho, resumo, cupom, forms e painel de sucesso;
- aplicação de cupom, recálculo de total e validação de carrinho vazio;
- checkout com RHF + Zod;
- restrição de pagamento no mesmo dia;
- aceite obrigatório da política;

- persistência de pedido mock.

Resultado esperado

- checkout completo e navegável;
- validações funcionando;
- pedido mock criado com sucesso.

Fase 5 — Conta, pedidos, FAQ refinado e política final

Objetivo da fase: concluir a experiência pós-compra e o conteúdo institucional final.

Entregas principais

- rotas de conta e pedidos;
- login e cadastro mock;
- listagem e detalhe de pedidos;
- refinamento de FAQ e política comercial;
- persistência local de sessão, carrinho e pedidos.

Resultado esperado

- área do cliente funcional;
- histórico de pedidos visível;
- conteúdo institucional consolidado.

Fase 6 — Testes, refinamento e README

Objetivo da fase: finalizar qualidade, testes e documentação.

Entregas principais

- testes unitários para pricing, availability, cart, checkout, stores e validators;
- testes de integração para adição de produtos e fluxo de checkout;
- testes E2E com Playwright para principais cenários de compra;
- refinamento de UX, acessibilidade, estados e mensagens;
- README completo com setup, execução, testes, estrutura e decisões técnicas.

Resultado esperado

- projeto estável;
- testes cobrindo fluxos principais;
- README pronto;
- qualidade de demo executiva.

Checklist de QA

QA funcional

- ☐ A landing page abre corretamente.
- ☐ A rota `/tickets` funciona.
- ☐ A rota `/water-park/p` funciona.
- ☐ O calendário mostra ao menos 2 meses.
- ☐ Datas passadas estão bloqueadas.
- ☐ Datas sold out estão bloqueadas.
- ☐ Selecionar data altera preços visíveis.
- ☐ O resumo lateral atualiza ao selecionar itens.
- ☐ Cards estão separados por categoria.
- ☐ "Saiba mais" expande corretamente.
- ☐ Stepper incrementa e decrementa corretamente.
- ☐ Ingresso simples pode ser adicionado ao carrinho.
- ☐ Passaporte 2 dias exige 2 datas.
- ☐ Passaporte 10 dias exige 10 datas.
- ☐ Combo exige datas dos dois parques.
- ☐ Ingresso especial mostra elegibilidade.
- ☐ Cabana sem ingresso principal gera erro.
- ☐ Minicart abre e fecha corretamente.
- ☐ Carrinho lista itens corretamente.
- ☐ Cupom válido altera total.

- ☐ Cupom inválido mostra erro.
- ☐ Carrinho vazio não segue para checkout.
- ☐ `/checkout/identify` funciona.
- ☐ `/checkout/customer` valida formulário.
- ☐ `/checkout/payment` mostra métodos corretamente.
- ☐ Compra para hoje restringe a pagamento Pix.
- ☐ `/checkout/review` exige aceite da política.
- ☐ `/checkout/success` exibe pedido criado.
- ☐ `/account/login` funciona em modo mock.
- ☐ `/account/register` funciona em modo mock.
- ☐ `/account/orders` lista pedidos.
- ☐ `/account/orders/[id]` mostra detalhes.
- ☐ `/faq` carrega busca e accordions.
- ☐ `/purchase-policy` carrega documento completo.

QA visual e UX

- ☐ Layout é original e não remete a cópia literal.
- ☐ Header está consistente em desktop e mobile.
- ☐ Footer está consistente.
- ☐ Sidebar sticky funciona em desktop.
- ☐ Mobile adapta layout corretamente.
- ☐ Estados loading existem.
- ☐ Estados empty existem.
- ☐ Estados error existem.
- ☐ Disabled states são claros.
- ☐ Toasts aparecem corretamente.
- ☐ Textos estão legíveis.
- ☐ Contraste está adequado.

- ☐ Navegação está clara.
- ☐ Interações têm feedback visual.
- ☐ Animações são suaves e discretas.

QA técnico

- ☐ TypeScript strict sem erros graves.
- ☐ ESLint sem erros críticos.
- ☐ Serviços desacoplados da UI.
- ☐ Stores separadas por contexto.
- ☐ Validações com RHF + Zod.
- ☐ Estado persiste em localStorage quando necessário.
- ☐ Mocks carregam corretamente.
- ☐ Projeto inicia sem quebra.
- ☐ Rotas navegam sem erro.
- ☐ README existe e está completo.

Critérios de Playwright

Suite 1 — `buy-single-day.spec.ts`

1. acessar `/water-park/p` ;
2. selecionar data disponível;
3. adicionar 1 ingresso adulto;
4. abrir minicart;
5. ir para `/cart` ;
6. avançar para `/checkout/identify` ;
7. preencher e-mail;
8. avançar para `/checkout/customer` ;
9. preencher dados válidos;
10. avançar para `/checkout/payment` ;

11. selecionar Pix;
12. avançar para `/checkout/review` ;
13. aceitar política;
14. finalizar pedido;
15. validar `/checkout/success` ;
16. validar número do pedido visível.

Suite 2 — `buy-passport-2-days.spec.ts`

1. acessar `/water-park/p` ;
2. selecionar 2 datas válidas;
3. adicionar passaporte 2 dias;
4. validar item no carrinho com 2 datas;
5. concluir checkout;
6. validar sucesso.

Suite 3 — `buy-combo.spec.ts`

1. acessar `/water-park/p` ;
2. selecionar data do parque principal;
3. selecionar data do parque complementar;
4. adicionar combo;
5. validar carrinho;
6. concluir checkout;
7. validar sucesso.

Suite 4 — `cabana-requires-ticket.spec.ts`

1. acessar `/water-park/p` ;
2. tentar adicionar cabana sem ingresso principal;
3. validar mensagem de erro;

4. validar que o item não foi adicionado.

Suite 5 — `same-day-payment-restriction.spec.ts`

1. acessar `/water-park/p` ;
2. selecionar a data de hoje;
3. adicionar ingresso;
4. seguir checkout;
5. chegar em `/checkout/payment` ;
6. validar que apenas Pix está disponível;
7. validar cartão indisponível;
8. validar wallet indisponível.

Suite 6 — `invalid-coupon.spec.ts`

1. adicionar item ao carrinho;
2. inserir cupom inválido;
3. validar toast de erro;
4. validar total sem alteração.

Suite 7 — `account-orders.spec.ts`

1. concluir uma compra mock;
2. acessar `/account/orders` ;
3. validar pedido listado;
4. abrir `/account/orders/[id]` ;
5. validar detalhes completos.

Seed JSON inicial — base operacional

Os arquivos abaixo formam a base inicial de mocks do projeto e podem ser ajustados conforme a estratégia comercial ou os cenários de teste.

Arquivos-base

- `parks.json`
- `categories.json`
- `products.json`
- `availability.json`
- `coupons.json`
- `users.json`
- `faq.json`
- `policy.json`

Cobertura esperada dos seeds

- `parks.json` : parque principal AquaVista Water Park e parque complementar Bosque Aventura, com descrição curta, hero e benefícios.
- `categories.json` : categorias para 1 dia, passaportes, combos, especiais e cabanas.
- `products.json` : produtos de adulto, criança, promo flex, passaporte 2 dias, passaporte 10 dias, combos, ingressos especiais e cabanas com regras de data, stock e limite por pedido.
- `availability.json` : datas com níveis `lowest`, `standard`, `peak`, `sold_out` e `low_availability`, incluindo preço-base e desconto Pix.
- `coupons.json` : cupons percentuais e fixos como `AQUA10`, `VIP20` e `FAMILY50`.
- `users.json` : usuário mock com pedidos, itens, datas selecionadas, instruções de visita e status de pagamento.
- `faq.json` : perguntas sobre compra, datas e passaportes.
- `policy.json` : condições gerais, preços, uso do ingresso, passaportes e combos multiparque.

Diretriz de modelagem dos mocks

- Os produtos devem refletir regras reais de data obrigatória, limite por pedido, estoque por data e elegibilidade.
- Cabanas devem depender de ingresso principal no carrinho.

- Combos devem guardar vínculo com parques relacionados.
- O seed deve permitir cenários felizes e cenários de erro para Playwright e QA manual.

Nota de amarração da Versão 4

Esta base consolidada deve ser lida em conjunto com:

- o **complemento de mapas, rotas e catálogo de destinos**;
- o **seed adicional** `park-destinations.json` ;
- o **handoff operacional para Replit/Cursor**, quando a implementação migrar da baseline funcional em Next.js para a baseline técnica em Vite + wouter + Express.

Complemento V4 — Mapas, Rotas e Catálogo de Destinos

Documento complementar para o plano AquaVista Tickets / módulo de ingressos e descoberta de parques.

Escopo: arquitetura de mapas dual, navegação por rotas, correção conceitual sobre BeiDou, integração com Google Maps e estrutura editorial-operacional para parques.

Decisão arquitetural recomendada

Usar uma stack própria de mapa aberto para exibição, descoberta e cálculo básico de rota dentro da plataforma, combinada com integração com Google Maps para navegação externa e rota assistida ao destino.

BeiDou entra apenas como fonte de posicionamento GNSS do dispositivo quando disponível, e não como plataforma open source de mapas.

Síntese executiva

O módulo de mapas deve cumprir quatro objetivos:

1. Mostrar o parque no mapa dentro da plataforma.
2. Permitir que o cliente trace rotas até o local.
3. Manter uma camada editorial própria por destino, sem desviar o tráfego para canais oficiais dos operadores.

4. Sustentar o módulo de ingressos com dados geográficos, horários, categorias, mídia e status interno.

Arranjo dual recomendado:

- **Mapa interno aberto** com MapLibre GL JS ou Leaflet para renderização e OpenStreetMap como base cartográfica, com mecanismo próprio de roteamento.
- **Integração paralela com Google Maps** para abrir rota externa quando o usuário quiser navegação nativa no app ou no navegador.

Esse desenho preserva independência da plataforma, reduz fricção e mantém descoberta, comparação, mapa e checkout no mesmo funil.

Correção conceitual: onde o BeiDou entra

BeiDou (BDS) é um sistema de navegação por satélite, equivalente conceitualmente a outros sistemas GNSS. Ele é útil para posicionamento do usuário em dispositivos compatíveis, mas não substitui:

- biblioteca de mapa web;
- provedor de tiles;
- motor de roteamento;
- plataforma de direções.

No plano técnico:

- BeiDou deve ser tratado como fonte de posicionamento do dispositivo, junto com GPS/GNSS do sistema operacional.
- O front web continua precisando de uma stack de mapas (MapLibre ou Leaflet).
- As rotas continuam precisando de um motor de directions/routing.
- A experiência de “como chegar” pode coexistir com Google Maps por deep link ou URLs de direções.

Arquitetura recomendada de mapas e rotas

Camada A — Mapa interno da plataforma

- Biblioteca: **MapLibre GL JS** (preferencial) ou **Leaflet**.

- Dados/cartografia: **OpenStreetMap** e/ou provedor compatível baseado em OSM.
- Exibição: marcadores, cards, clusters, filtros por categoria, mapa por cidade e mapa por destino.
- UI: modal de rota, botão “Como chegar”, mini mapa na PDP do parque e mapa full screen em `/destinations/map`.

Camada B — Motor de rota dentro da plataforma

- Opção recomendada: **OSRM** para rota de carro, caminhada e bicicleta em dados OSM.
- Alternativas futuras: **Valhalla**, **GraphHopper** ou serviço comercial baseado em OSM.
- Saída esperada: distância, duração estimada, polyline, passos resumidos, rota principal e, se desejado, rota alternativa.

Camada C — Navegação externa

- Botão “Abrir no Google Maps”.
- Botão “Traçar rota no Google Maps”.
- Deep links com origem do usuário e destino do parque.

Camada D — Posicionamento do usuário

- Geolocation do navegador/app.
- Quando o dispositivo suportar, o posicionamento pode vir do stack GNSS do aparelho, incluindo sinais como GPS e BeiDou, de forma transparente para o sistema web.

Regras funcionais do módulo de rotas

O módulo deve entregar:

- detecção da localização atual do usuário mediante consentimento;
- fallback por endereço digitado, CEP ou ponto selecionado no mapa;
- cálculo de rota até o parque com pelo menos modo carro no MVP;

- exibição de distância e tempo estimado;
- botão para iniciar rota interna na plataforma;
- botão para abrir rota no Google Maps;
- armazenamento do destino como atributo do parque;
- pré-cálculo opcional de rotas populares para páginas de destino;
- uso do mapa também em páginas de cidade, categoria e comparação.

Regras de UX

- Nunca obrigar o usuário a sair do site para entender onde fica o parque.
- Só abrir o Google Maps quando a intenção for navegação externa.
- Em mobile, priorizar botões claros: "Ver no mapa", "Traçar rota", "Abrir no Google Maps".
- Em desktop, exibir mini mapa + CTA de rota + endereço legível + coordenadas internas para analytics/logística.

Restrições e cuidados

- Em produção, não usar os servidores públicos padrão do OpenStreetMap como backend principal de alto tráfego.
- Preferir tiles próprios, cache próprio ou provedor comercial compatível com OSM.
- No lado Google, APIs/SDKs de mapas e rotas exigem autenticação e faturamento.
- Para apenas abrir o Google Maps com um link de rota, usar Maps URLs sem chave quando apropriado.

Separação recomendada:

- **rota interna e mapa próprio** = stack aberto / plataforma própria;
- **navegação externa** = Google Maps por link de rota.

Impacto no funil de ingressos

O módulo de mapas deve reforçar o funil comercial:

- na página do parque: endereço, mapa, rota, bairros próximos, tempo estimado e contexto da visita;
- no módulo de ingressos: destino no mapa + CTA "Planejar chegada";
- no carrinho/checkout: reforço de localização e dia da visita;
- no pós-venda: rota rápida, instruções de acesso, estacionamento, entrada principal e observações da visita;
- em destino/cidade: descoberta e comparação entre parques.

Objetivos comerciais:

- reduzir abandono por dúvida de localização;
- aumentar confiança do usuário;
- melhorar percepção de conveniência;
- preparar base para automações de pré-visita e lembretes geográficos.

Estrutura de dados adicional

Adicionar um novo domínio para suportar mapas, rota e descoberta editorial.

Entidade principal sugerida: **ParkDestination**

Campos essenciais

- `id` , `slug` , `name` , `city` , `state` , `country`
- `address` , `latitude` , `longitude`
- `placeType`
- `summary`
- `attractions[]`
- `openingHours.regular` , `openingHours.notes` , `openingHours.seasonalRules[]`
- `operationalStatus`
- `editorialCategory`
- `internalConfidenceStatus`
- `mediaGallery.coverImage` , `galleryImages[]` , `shortVideos[]`
- `platformPresence.profileSlug` , `featured` , `priority` , `tags[]`

- `ticketingRelations.hasTickets` , `relatedProducts[]`
- `contentBlocks.heroTitle` , `heroSubtitle` , `sellingPoints[]` , `visitorNotes[]`

Campos de rota

- `routeProfileDefault: driving | walking | cycling`
- `parkingInfo`
- `entryPointLabel`
- `entryPointLatitude`
- `entryPointLongitude`
- `accessNotes[]`
- `nearbyLandmarks[]`
- `routeEnabled`
- `googleMapsEnabled`
- `internalRoutingEnabled`
- `geofenceRadiusMeters`

Campos de analytics

- `mapViews`
- `routeRequests`
- `openInGoogleMapsClicks`
- `userLocationPermissionRate`
- `destinationSaveRate`

Rotas e páginas novas

Adicionar à Versão 4:

- `/destinations`
- `/destinations/[slug]`
- `/destinations/category/[category]`
- `/destinations/city/[city]`
- `/destinations/map`

- `/route?destination=[slug]`
- `/route/preview/[slug]`

Essas páginas devem suportar:

- lista e filtros;
- mapa de todos os parques;
- mapa por cidade;
- página do parque com mini mapa;
- preview de rota;
- abrir em Google Maps;
- relacionar destino com produtos de ingresso.

Componentes adicionais

- `DestinationCard`
- `DestinationHero`
- `DestinationAttractions`
- `DestinationHighlights`
- `DestinationOperationalBadge`
- `DestinationEditorialTags`
- `DestinationMediaGallery`
- `DestinationVideoRail`
- `DestinationMapBlock`
- `DestinationRoutePlanner`
- `DestinationRouteSummary`
- `DestinationOpenInGoogleButton`
- `RelatedTicketsSection`
- `DestinationComparisonCard`
- `MapProviderSwitcher`
- `RouteModeSelector`

- `UserLocationPrompt`
- `NearbyParksMapRail`

Critérios de aceite complementares

O complemento de mapas só estará pronto se:

- cada parque possuir latitude e longitude válidas;
- existir pelo menos uma experiência de mapa dentro da plataforma;
- existir botão de abrir rota externa no Google Maps;
- a rota interna suportar pelo menos carro no MVP;
- os dados de destino alimentarem também as páginas do módulo de ingressos;
- o sistema não expuser telefone, site ou Instagram dos operadores no front;
- a plataforma usar sua própria camada editorial e sua própria mídia;
- o mapa estiver funcional em desktop e mobile;
- houver fallback quando o usuário não liberar localização;
- existir estratégia de produção que não dependa do tile server público padrão da OSMF.

Prompt pronto para anexar à Versão 4



Implemente um módulo dual de mapas. Use MapLibre GL JS ou Leaflet com dados/cartografia compatíveis com OpenStreetMap para o mapa interno da plataforma. Use um motor de roteamento dedicado para cálculo de rotas internas. Adicione integração com Google Maps via link/URL de direções para navegação externa. Trate BeiDou apenas como fonte de posicionamento GNSS do dispositivo quando disponível, e não como plataforma de mapa ou roteamento. O usuário deve conseguir ver o parque no mapa, obter rota interna resumida e também abrir o trajeto no Google Maps. Não exibir telefone, Instagram ou site do operador do parque. Toda presença editorial e mídia deve ser da plataforma.

Campos recomendados para o módulo de mapas no catálogo de parques

Grupo	Campo	Uso	Obrigatório
Identidade	<code>id</code> / <code>slug</code> / <code>name</code>	Identificação interna, URL e título público	Sim
Localização	<code>city</code> / <code>state</code> / <code>country</code>	Filtro por cidade e destino	Sim
Endereço	<code>address</code>	Exibição e ponto de navegação	Sim
Coordenadas	<code>latitude</code> / <code>longitude</code>	Mapa, rota e analytics	Sim
Classificação	<code>placeType</code> / <code>editorialCategory</code>	Vitrine, filtros e recomendação	Sim
Conteúdo	<code>summary</code> / <code>attractions[]</code>	Página do parque e módulos editoriais	Sim
Operação	<code>openingHours</code> / <code>operationalStatus</code>	Contexto da visita e confiança	Sim
Editorial	<code>mediaGallery</code> / <code>contentBlocks</code>	Conteúdo próprio da plataforma	Sim
Comercial	<code>ticketingRelations</code>	Conectar destino ao módulo de ingressos	Sim
Rota	<code>routeProfileDefault</code> / <code>entryPoint</code> / <code>accessNotes</code>	Planejamento de chegada	Sim
Analytics	<code>mapViews</code> / <code>routeRequests</code> / <code>openInGoogleMapsClicks</code>	Medição de uso do módulo	Não

Exemplo resumido de JSON para destinos

```
{
  "id": "dest-hot-park",
  "slug": "hot-park",
  "name": "Hot Park",
  "city": "Rio Quente",
  "state": "GO",
  "country": "Brasil",
```

```
"address": "Complexo turístico, Rio Quente - GO",
"latitude": -17.770949,
"longitude": -48.75338,
"placeType": "water_park",
"editorialCategory": "mixed",
"openingHours": {
  "regular": "10:00 às 17:00",
  "notes": "Revisar operação sazonal"
},
"routeEnabled": true,
"googleMapsEnabled": true,
"internalRoutingEnabled": true,
"ticketingRelations": {
  "hasTickets": false,
  "relatedProducts": []
}
}
```

Fontes de referência técnica

- BeiDou official overview / BDS: sistema de navegação por satélite, não plataforma de mapas web.
- MapLibre GL JS: biblioteca open source para mapas no navegador.
- Leaflet: biblioteca open source para mapas interativos.
- OSRM: motor open source de roteamento sobre dados OpenStreetMap.
- Google Maps URLs: permitem abrir Google Maps para direções sem chave de API.
- Google Routes / Directions docs: APIs de rota da Google exigem autenticação e faturamento.
- OSMF Tile Usage Policy: dados OSM são livres, mas os servidores públicos de tiles têm política de uso e capacidade limitada.

Plano de Implantação (Cursor AI + Replit) — Passo a passo por fases

Este plano é o **guia principal** para o Cursor AI e para o Replit implementarem o AquaVista Tickets como um projeto executável, com **código-base pronto** (scaffold) e uma sequência robusta de execução.

Objetivo

- Construir uma aplicação completa (front + backend mock) com **Vite + React + TypeScript + wouter** no client e **Express** no server.
- Usar `shared/` para tipos, validators e seeds.
- Garantir que o sistema consiga rodar e ser validado por QA manual + Playwright.

Stack obrigatória (baseline de implantação)

- Frontend: Vite + React 18 + TypeScript strict + wouter
 - UI: Tailwind + shadcn/ui + lucide-react
 - Estado assíncrono: TanStack Query v5
 - Estado local: Zustand
 - Forms: React Hook Form + Zod
 - Backend: Express
 - Testes: Vitest + Testing Library + Playwright
-

Fase 0 — Preparação do repositório e convenções

Etapas 0.1 — Estrutura de pastas (monorepo simples)

Crie a estrutura abaixo (pasta-raiz do projeto):

- `client/` (Vite)
- `server/` (Express)
- `shared/` (types, validators, seeds)
- `tests/` (unit/integration/e2e)

Etapas 0.2 — Convenções

- Linguagem do produto: pt-BR
- Moeda: BRL

- Datas: ISO (dados) + exibição pt-BR
- Rotas do front conforme especificação já listada neste documento

Etapa 0.3 — Variáveis de ambiente

- `client/.env.example` e `server/.env.example`
- No client use `import.meta.env`.

Fase 1 — Código-base pronto (scaffold) para rodar no Replit/Cursor

Esta fase cria um projeto que já inicia com `client` e `server` funcionando.

Etapa 1.1 — package.json (raiz)

Crie um `package.json` na raiz com scripts para rodar tudo.

Código sugerido (ajuste versões conforme necessidade do ambiente):

```
{
  "name": "aquavista-tickets",
  "private": true,
  "workspaces": ["client", "server", "shared"],
  "scripts": {
    "dev": "concurrently -n client,server \"npm:dev:client  
\" \"npm:dev:server\"",
    "dev:client": "npm --prefix client run dev",
    "dev:server": "npm --prefix server run dev",
    "build": "npm --prefix client run build && npm --prefix  
server run build",
    "test": "npm --prefix client run test && npm --prefix s  
erver run test",
    "lint": "npm --prefix client run lint && npm --prefix s  
erver run lint"
  },
  "devDependencies": {
    "concurrently": "^9.0.0"
  }
}
```

Etapa 1.2 — shared/: tipos, validators e seeds

1.2.1 shared/types (domínio)

Crie `shared/src/types/domain.ts` com os tipos mínimos para destravar o projeto.

Código-base sugerido:

```
export type Park = {
  id: string
  slug: string
  name: string
  shortDescription: string
  heroImage: string
  benefits: string[]
}

export type ProductCategory = {
  id: string
  slug: string
  title: string
  description: string
  displayOrder: number
}

export type EligibilityRule = {
  code: string
  label: string
  description: string
}

export type TicketProduct = {
  id: string
  slug: string
  parkId: string
  categoryId: string
  type: "single_day" | "passport" | "combo" | "special" |
  "cabana"
  audience: "adult" | "child" | "mixed" | "senior" | "pcd"
```

```

| "pregnant"
  title: string
  shortDescription: string
  longDescription: string
  benefits: string[]
  rules: string[]
  basePrice: number
  pixPrice: number
  creditPrice: number
  requiresDateSelection: boolean
  requiredDatesCount: number
  linkedParkIds: string[]
  eligibilityRules: EligibilityRule[]
  maxPerOrder: number
  stockMode: "by_date" | "by_capacity"
  capacity?: number
  active: boolean
  tags: string[]
}

export type AvailabilityDay = {
  date: string
  priceLevel: "lowest" | "standard" | "peak" | "sold_out"
  status: "available" | "sold_out" | "low_availability"
  basePriceAdult: number
  basePriceChild: number
  pixDiscountPercent: number
  tags: string[]
}

export type Coupon = {
  code: string
  type: "percentage" | "fixed"
  value: number
  active: boolean
}

export type PaymentMethod = "pix" | "card" | "wallet"

```



```
export type PaymentStatus = "pending" | "approved" | "failed"

export type OrderItem = {
  id: string
  productId: string
  productType: TicketProduct["type"]
  title: string
  quantity: number
  selectedDates: string[]
  selectedParks: string[]
  unitPrice: number
  pixUnitPrice: number
  subtotal: number
}

export type Order = {
  id: string
  number: string
  paymentMethod: PaymentMethod
  paymentStatus: PaymentStatus
  policyAccepted: boolean
  createdAt: string
  total: number
  visitInstructions: string[]
  items: OrderItem[]
}

export type User = {
  id: string
  firstName: string
  lastName: string
  email: string
  phone: string
  document: string
  orders: Order[]
}
```

1.2.2 shared/validators (Zod)

Crie `shared/src/validators/checkout.ts` com um schema mínimo do cliente.

```
import { z } from "zod"

export const identifySchema = z.object({
  email: z.string().email(),
})

export const customerSchema = z.object({
  firstName: z.string().min(2),
  lastName: z.string().min(2),
  document: z.string().min(11),
  phone: z.string().min(10),
})
```

1.2.3 shared/seeds

Crie `shared/src/seeds/` e coloque os arquivos JSON (os mesmos já documentados nesta página):

- `parks.json`
- `categories.json`
- `products.json`
- `availability.json`
- `coupons.json`
- `users.json`
- `faq.json`
- `policy.json`

Fase 2 — Backend Express (mock robusto)

Etapas 2.1 — server bootstrap

Crie `server/src/index.ts`.

```

import express from "express"
import cors from "cors"

import { catalogRouter } from "../routes/catalog"
import { availabilityRouter } from "../routes/availability"
import { pricingRouter } from "../routes/pricing"
import { checkoutRouter } from "../routes/checkout"
import { ordersRouter } from "../routes/orders"

const app = express()
app.use(cors())
app.use(express.json())

app.get("/health", (_req, res) => res.json({ ok: true }))

app.use("/api/catalog", catalogRouter)
app.use("/api/availability", availabilityRouter)
app.use("/api/pricing", pricingRouter)
app.use("/api/checkout", checkoutRouter)
app.use("/api/orders", ordersRouter)

const port = Number(process.env.PORT ?? 3001)
app.listen(port, () => {
  console.log(`[server] listening on :${port}`)
})

```

Etapa 2.2 — Rotas do server (mínimo funcional)

2.2.1 /api/catalog

server/src/routes/catalog.ts

```

import { Router } from "express"
import parks from "../../shared/src/seeds/parks.json"
import categories from "../../shared/src/seeds/categories.json"
import products from "../../shared/src/seeds/products.json"

```

```
export const catalogRouter = Router()

catalogRouter.get("/", (_req, res) => {
  res.json({ parks, categories, products })
})
```

2.2.2 /api/availability

server/src/routes/availability.ts

```
import { Router } from "express"
import availability from "../../shared/src/seeds/availability.json"

export const availabilityRouter = Router()

availabilityRouter.get("/", (_req, res) => {
  res.json({ availability })
})
```

2.2.3 /api/pricing (regra por data)

server/src/routes/pricing.ts

```
import { Router } from "express"
import availability from "../../shared/src/seeds/availability.json"

export const pricingRouter = Router()

pricingRouter.get("/", (req, res) => {
  const date = String(req.query.date ?? "")
  const day = (availability as any[]).find((d) => d.date === date)
  if (!day) return res.status(404).json({ message: "date_not_found" })
})
```

```
    return res.json({ date, day })
  })
```

2.2.4 /api/checkout (criação de pedido mock)

server/src/routes/checkout.ts

```
import { Router } from "express"

export const checkoutRouter = Router()

checkoutRouter.post("/", (req, res) => {
  const payload = req.body
  // MVP: validação simplificada; evoluir usando shared/validators
  const orderNumber = `AV-${new Date().toISOString().slice(
    0, 10).replaceAll("-", "")}-${String(Math.floor(Math.random() * 9000) + 1000)}`

  return res.json({
    id: `order-${Date.now()}`,
    number: orderNumber,
    paymentStatus: "approved",
    ...payload,
  })
})
```

2.2.5 /api/orders (lista mock)

server/src/routes/orders.ts

```
import { Router } from "express"
import users from "../../shared/src/seeds/users.json"

export const ordersRouter = Router()

ordersRouter.get("/", (_req, res) => {
  const first = (users as any[][0])
```

```
res.json({ orders: first?.orders ?? [] })
})
```

Fase 3 — Frontend Vite + wouter (rotas e layout)

Etapas 3.1 — client bootstrap

Crie `client/src/main.tsx`.

```
import React from "react"
import ReactDOM from "react-dom/client"
import { QueryClient, QueryClientProvider } from "@tanstack/react-query"
import { AppRouter } from "../router"
import "../styles/globals.css"

const qc = new QueryClient()

ReactDOM.createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <QueryClientProvider client={qc}>
      <AppRouter />
    </QueryClientProvider>
  </React.StrictMode>
)
```

Etapas 3.2 — Router do front

`client/src/router.tsx`

```
import { Route, Switch } from "wouter"
import { HomePage } from "../pages/HomePage"
import { TicketsHubPage } from "../pages/TicketsHubPage"
import { WaterParkPage } from "../pages/WaterParkPage"
import { CartPage } from "../pages/CartPage"

export function AppRouter() {
```

```

return (
  <Switch>
    <Route path="/" component={HomePage} />
    <Route path="/tickets" component={TicketsHubPage} />
    <Route path="/water-park/p" component={WaterParkPage} />
  />
  <Route path="/cart" component={CartPage} />
  <Route>404</Route>
</Switch>
)
}

```

Etapa 3.3 — Service layer (client)

`client/src/services/api.ts`

```

export const API_BASE = import.meta.env.VITE_API_BASE ?? "http://localhost:3001"

export async function apiGet<T>(path: string): Promise<T> {
  const res = await fetch(`${API_BASE}${path}`)
  if (!res.ok) throw new Error(`HTTP ${res.status}`)
  return res.json() as Promise<T>
}

```

`client/src/services/catalog.ts`

```

import { apiGet } from "../api"

export type CatalogResponse = {
  parks: any[]
  categories: any[]
  products: any[]
}

export function getCatalog() {
  return apiGet<CatalogResponse>("/api/catalog")
}

```

Fase 4 — Catálogo, calendário e carrinho (regras críticas)

Nesta fase, implemente os módulos e valide as regras abaixo.

Etapa 4.1 — Regras obrigatórias (ordem de implementação)

1. Data obrigatória para preço final
2. Datas passadas bloqueadas
3. Datas sold out bloqueadas
4. Pix com preço preferencial (quando aplicável)
5. Compra para o mesmo dia restringe pagamento para Pix (no checkout)
6. Passaporte 2 dias exige 2 datas
7. Passaporte 10 dias exige 10 datas
8. Combo exige datas de ambos os parques
9. Cabana exige ingresso principal no carrinho
10. Cupom válido recalcula total

Etapa 4.2 — Store Zustand (cart)

Crie `client/src/store/useCartStore.ts`.

```
import { create } from "zustand"

type CartItem = {
  productId: string
  quantity: number
  selectedDates: string[]
  selectedParks: string[]
}

type CartState = {
  items: CartItem[]
  addItem: (item: CartItem) => void
  clear: () => void
}
```



```
export const useCartStore = create<CartState>((set) => ({
  items: [],
  addItem: (item) => set((s) => ({ items: [...s.items, item] })),
  clear: () => set({ items: [] }),
}))
```

Fase 5 — Checkout completo + pedidos

- Implementar as rotas de checkout em 5 etapas.
- Integrar com `/api/checkout` para gerar pedido mock.
- Persistir pedido (localStorage no MVP) e exibir em `/account/orders`.

Fase 6 — Testes (Vitest + Playwright)

Etapa 6.1 — Unit tests

- pricing/availability/cart/checkout

Etapa 6.2 — E2E Playwright (suites)

Use as suítes já descritas neste documento (single day, passport, combo, cabana restriction, same day restriction, invalid coupon, account orders).

Checkpoint de entrega (aceite de implantação)

Considere a implantação aprovada quando:

- `npm run dev` sobe client e server sem erro
- `/health` responde ok
- `/api/catalog` retorna parks/categories/products
- `/water-park/p` renderiza catálogo + calendário
- Regras críticas do carrinho e dos produtos estão funcionando
- Checkout completa e gera pedido
- Ao menos 1 suite Playwright passa end-to-end

Complemento V4 — Handoff para Replit + Cursor AI

AquaVista Tickets — Pacote de Execução para Replit + Cursor AI

Versão 4 — handoff consolidado, adaptação de stack, `.cursorrules` e plano operacional.

Objetivo: entregar um pacote pronto para o Replit analisar, adequar, testar e servir como base de implantação.

Mudança principal: converter a arquitetura da Versão 4 consolidada de Next.js para **Vite + wouter + Express**, sem perder lógica, rotas, cards, menus, animações e regras de negócio.

Status: pronto para execução assistida. Commit/push no GitHub dependem do ambiente autenticado do usuário.

Base de referência: a Versão 4 já reúne prompt mestre, fases, QA, seeds e o complemento de mapas, rotas e catálogo de destinos. Este handoff preserva esse escopo, mas o reorienta para a stack solicitada no Cursor/Replit.

Decisão arquitetural e restrições reais

A Versão 4 consolidada existente foi concebida em **Next.js 14 + App Router**, com foco em ticketing, catálogo editorial, mapas e rotas. Este documento adapta esse escopo para a stack pedida no Replit/Cursor: **Vite, wouter, TanStack Query v5, shadcn/ui, lucide-react e Express**.

Essa troca exige uma reinterpretação das rotas, do SSR/SPA e da organização de pastas, mas não altera o produto alvo.

Não é possível executar commit e push em um repositório GitHub a partir deste ambiente sem acesso autenticado ao repositório do usuário. O pacote deixa pronta a estrutura para execução local: regras, `.cursorrules`, checklist, rotas, critérios de teste e instruções de commit/push.

O complemento de mapas já definiu a arquitetura dual recomendada: mapa interno com stack aberta, rotas internas e abertura opcional no Google Maps. Também corrigiu a premissa sobre BeiDou: ele entra como GNSS/posicionamento do dispositivo, não como plataforma de mapas web ou engine de direções.

Stack final alvo para Replit + Cursor AI

- **Frontend:** Vite + React + TypeScript + wouter + TanStack Query v5 + shadcn/ui + lucide-react + react-hook-form + zod + framer-motion.

- **Backend:** Express + WebSocket broadcast para eventos internos simulados, com rotas REST para catálogo, disponibilidade, pricing, carrinho, checkout, pedidos, destinos e rotas.
- **Estado:** Zustand para stores locais e TanStack Query para cache/estado assíncrono.
- **Estilo:** tokens HSL, dark mode e semântica forte para sucesso, alerta, erro, info e premium.
- **Testes:** Vitest + Testing Library + Playwright.
- **Mapas e rotas:** MapLibre ou Leaflet + dados/cartografia compatíveis com OSM + integração por link com Google Maps; BeiDou apenas como fonte transparente de posicionamento quando o dispositivo suportar.

Migração conceitual da Versão 4 para a nova stack

- Substituir App Router do Next.js por registro explícito de rotas no frontend via wouter.
- Substituir routes API do Next por servidor Express separado, servindo endpoints mock e futuramente integrações reais.
- Manter o mesmo mapa de produto: landing, hub de ingressos, PDP do parque, carrinho, checkout em 5 etapas, conta, FAQ, política, destinos, cidades, mapa, rota e pós-venda.
- Transformar seeds JSON em shared data, reaproveitáveis no frontend e no backend.
- Preservar cards, stepper, minicart, menu, submenus, animações e regras comerciais descritas na Versão 4.

Estrutura de arquivos recomendada

```

/
├─ client/
│   └─ src/
│       └─ app/
│           ├── router.tsx
│           ├── providers.tsx
│           └─ layout/
│               └─ pages/

```

```

| | | └─ HomePage.tsx
| | | └─ TicketsHubPage.tsx
| | | └─ WaterParkPage.tsx
| | | └─ CartPage.tsx
| | | └─ checkout/
| | | └─ account/
| | | └─ destinations/
| | | └─ FaqPage.tsx
| | | └─ PurchasePolicyPage.tsx
| | └─ components/
| | └─ layout/
| | └─ catalog/
| | └─ cart/
| | └─ checkout/
| | └─ destinations/
| | └─ maps/
| | └─ feedback/
| | └─ ui/
| | └─ hooks/
| | └─ lib/
| | └─ services/
| | └─ store/
| | └─ styles/
| | └─ main.tsx
└─ server/
  └─ index.ts
  └─ routes/
  └─ services/
  └─ sockets/
  └─ utils/
└─ shared/
  └─ types/
  └─ constants/
  └─ seeds/
  └─ validators/
└─ tests/
  └─ unit/
  └─ integration/

```

```
|   └─ e2e/
|   └─ .cursorrules
|   └─ package.json
|   └─ README.md
```

Registro de rotas obrigatório

Públicas

- `/`
- `/tickets`
- `/faq`
- `/purchase-policy`

Ingressos

- `/water-park/p`
- `/cart`

Checkout

- `/checkout/identify`
- `/checkout/customer`
- `/checkout/payment`
- `/checkout/review`
- `/checkout/success`

Conta

- `/account/login`
- `/account/register`
- `/account/orders`
- `/account/orders/:id`

Destinos

- `/destinations`
- `/destinations/:slug`
- `/destinations/category/:category`
- `/destinations/city/:city`
- `/destinations/map`

Rotas

- `/route?destination=:slug`
- `/route/preview/:slug`

Menus e submenus sugeridos

- **Menu principal:** Ingressos, Destinos, Planeje sua Visita, Minha Conta, FAQ.
- **Submenu de Destinos:** Por cidade, Por categoria, Mapa, Destaques.
- **Submenu de Planeje sua Visita:** Como chegar, Horários, Regras de uso, Acesso e estacionamento.
- **Minicart** fixo no header com estado vazio, resumo e CTA para carrinho.

Regras de negócio e lógica que não podem ser perdidas

- Data obrigatória para cálculo de preço final.
- Dias passados bloqueados.
- Dias sold out bloqueados.
- Preço dinâmico por data.
- Compra para o mesmo dia restringe pagamento a Pix.
- Passaporte 2 dias exige 2 datas; passaporte 10 dias exige 10 datas.
- Combos exigem datas dos dois parques.
- Cabanas exigem ingresso principal no carrinho.
- Minicart, carrinho e resumo sticky recalculam automaticamente.
- FAQ, política, destinos e mapas também devem alimentar o pós-venda.

- Não exibir telefone, site ou Instagram do operador; usar presença editorial própria.

Animações, efeitos e padrões visuais

- Framer Motion para fade-in, expand/collapse, drawer do minicart, transição de etapas e banners informativos.
- Hover com elevação leve em cards, badges e CTAs.
- Skeleton loaders em calendário, catálogo, destinos e mapas.
- Sticky summary suave em desktop e versão compacta em mobile.
- Dark mode obrigatório e consistente com tokens HSL.
- `data-testid` nos elementos críticos do funil para Playwright e regressão visual.

Backend Express e WebSocket

- Rotas REST: `/api/catalog` , `/api/availability` , `/api/pricing` , `/api/cart` , `/api/checkout` , `/api/orders` , `/api/destinations` , `/api/routes` .
- WebSocket broadcast para eventos simulados: atualização de carrinho, refresh de disponibilidade, mudança de status de pedido e notificações internas.
- Separar camada de service da camada de route; nunca codificar regra de negócio complexa diretamente no handler.
- Manter `shared/types` e `shared/validators` reaproveitados pelo client e pelo server.

Fluxo operacional passo a passo para execução

1. Criar branch de trabalho e conectar o projeto ao GitHub no Replit/Cursor.
2. Aplicar a nova stack base em Vite + React + TypeScript + Express.
3. Criar `shared/types` , `shared/seeds` e `shared/validators` a partir da Versão 4 consolidada.
4. Implementar design system, layout global, header, footer, menus e dark mode.
5. Implementar páginas públicas e hub de ingressos.

6. Implementar `/water-park/p` com calendário, cards, minicart e resumo sticky.
7. Implementar carrinho, checkout, pedidos e conta.
8. Implementar destinos, mapa, rota interna e botão “Abrir no Google Maps”.
9. Executar testes unitários, integração e E2E.
10. Somente após aprovação, fazer commit e push no repositório autenticado.

Instruções reais sobre commit e push

Não é possível executar push para GitHub a partir deste ambiente sem autenticação e sem acesso ao repositório.

Procedimento correto no Replit/Cursor:

1. Abrir o projeto com o repositório conectado.
2. Executar testes e lint.
3. Confirmar os arquivos alterados, inclusive o `.cursorrules`.
4. Criar commit com mensagem padronizada.
5. Executar push na branch alvo ou abrir Pull Request.

Mensagem sugerida de commit:

```
feat: aquavista vite-wouter-express handoff, cursorrules, routes, maps and ticketing workflow
```

Conteúdo recomendado para o arquivo `.cursorrules`

Projeto: AquaVista Tickets

STACK OBRIGATÓRIA

- Frontend: Vite + React + TypeScript + wouter
- UI: shadcn/ui + lucide-react + Tailwind
- Estado assíncrono: TanStack Query v5
- Estado local/global: Zustand
- Formulários: React Hook Form + Zod
- Backend: Express
- Testes: Vitest + Testing Library + Playwright
- Animações: Framer Motion

PADRÃO DE ARQUITETURA

- Separar client, server e shared
- Rotas do frontend registradas em wouter
- Endpoints REST no Express
- Regras de negócio em services, nunca espalhadas nos componentes
- Seeds, tipos e validators em shared
- Todos os fluxos devem ser pt-BR
- Usar data-testid nos elementos críticos

DESIGN SYSTEM

- Usar tokens HSL
- Semântica obrigatória: success, warning, error, info, premium
- Manter dark mode funcional
- Cards, botões, badges e inputs devem seguir o mesmo padrão visual
- Hero sections e CTA sections precisam manter espaçamento amplo e hierarquia clara

ROTAS OBRIGATÓRIAS

- /
- /tickets
- /water-park/p
- /cart
- /checkout/identify
- /checkout/customer
- /checkout/payment
- /checkout/review
- /checkout/success
- /account/login
- /account/register
- /account/orders
- /account/orders/:id
- /faq
- /purchase-policy
- /destinations
- /destinations/:slug
- /destinations/category/:category

- /destinations/city/:city
- /destinations/map
- /route
- /route/preview/:slug

COMPONENTES OBRIGATÓRIOS

- Header, Footer, MainNavigation, MobileMenu
- ProductCard, QuantityStepper, StickyTripSummary, MiniCart Drawer
- CheckoutStepper, IdentifyStepForm, CustomerStepForm, PaymentStepSelector, ReviewStepPanel
- DestinationCard, DestinationHero, DestinationMapBlock, DestinationRoutePlanner
- LoadingSkeleton, EmptyState, ErrorState, ToastFeedback

REGRAS DE CÓDIGO

- Usar TanStack Query v5 corretamente, sem APIs antigas
- Não usar Next.js
- Não usar process.env no client; usar import.meta.env
- Não importar React desnecessariamente em arquivos com JSX moderno
- Não misturar fetch solto com camada de service
- Não remover data-testid dos fluxos críticos
- Não acoplar lógica de pricing diretamente no JSX
- Não usar any sem justificativa real
- Não quebrar dark mode
- Não criar componentes gigantes sem composição
- Não deixar strings comerciais em inglês no front final
- Não expor canais externos dos parques no front
- Não substituir mapa interno apenas por link externo
- Não ignorar responsividade
- Não remover cobertura de testes dos fluxos principais

BACKEND

- Express com rotas organizadas por domínio
- WebSocket broadcast para eventos simulados de carrinho, disponibilidade e pedidos
- Validar payloads com Zod sempre que aplicável

CONTEÚDO E FORMATAÇÃO

- Idioma pt-BR
- Datas em padrão brasileiro
- Moeda em real brasileiro
- CTA claros e consistentes

NAVEGAÇÃO CIRCULAR

- Qualquer novo item de menu deve ser registrado em `constants/navigation.ts`
- Toda rota nova deve ter link reverso para páginas pai quando fizer sentido
- Destinos devem apontar para ingressos relacionados e ingressos devem apontar para destino

MAPAS E ROTAS

- Mapa interno com stack aberta
- Google Maps apenas como opção de rota externa
- BeiDou apenas como referência GNSS do dispositivo, nunca como plataforma de mapas

Atualização documental recomendada

Ao atualizar o documento principal da Versão 4, incorporar esta nota de governança:



A implementação efetiva para Replit/Cursor passa a usar **Vite + wouter + Express** como stack operacional. A documentação anterior em Next.js permanece como referência funcional e de escopo, mas não como baseline de código.

Conclusão prática

É possível executar o projeto completo de forma detalhada, arquitetada e robusta para Replit/Cursor, desde que a equipe trate a Versão 4 consolidada como especificação funcional e este handoff como baseline técnica de implementação.

O único ponto que permanece dependente do ambiente autenticado do usuário é commit/push no GitHub. Todo o restante — inclusive `.cursorrules`, rotas, menus, regras, mapas, QA e roteiro de execução — fica encapsulado neste pacote.

Checklist de QA — Versão 4

Versão 4 separada para execução no Energent

AquaVista Tickets • pacote documental

Checklist pronto para revisão manual, homologação interna e acompanhamento de entrega.

QA funcional

- ☐ A landing page abre corretamente
- ☐ A rota `/tickets` funciona
- ☐ A rota `/water-park/p` funciona
- ☐ O calendário mostra ao menos 2 meses
- ☐ Datas passadas estão bloqueadas
- ☐ Datas sold out estão bloqueadas
- ☐ Selecionar data altera preços visíveis
- ☐ O resumo lateral atualiza ao selecionar itens
- ☐ Cards estão separados por categoria
- ☐ "Saiba mais" expande corretamente
- ☐ Stepper incrementa e decrementa corretamente
- ☐ Ingresso simples pode ser adicionado ao carrinho
- ☐ Passaporte 2 dias exige 2 datas
- ☐ Passaporte 10 dias exige 10 datas
- ☐ Combo exige datas dos dois parques
- ☐ Ingresso especial mostra elegibilidade
- ☐ Cabana sem ingresso principal gera erro
- ☐ Minicart abre e fecha corretamente

- ☐ Carrinho lista itens corretamente
- ☐ Cupom válido altera total
- ☐ Cupom inválido mostra erro
- ☐ Carrinho vazio não segue para checkout
- ☐ `/checkout/identify` funciona
- ☐ `/checkout/customer` valida formulário
- ☐ `/checkout/payment` mostra métodos corretamente
- ☐ Compra para hoje restringe a pagamento Pix
- ☐ `/checkout/review` exige aceite da política
- ☐ `/checkout/success` exibe pedido criado
- ☐ `/account/login` funciona em modo mock
- ☐ `/account/register` funciona em modo mock
- ☐ `/account/orders` lista pedidos
- ☐ `/account/orders/[id]` mostra detalhes
- ☐ `/faq` carrega busca e accordions
- ☐ `/purchase-policy` carrega documento completo

QA visual e UX

- ☐ Layout é original e não remete a cópia literal
- ☐ Header está consistente em desktop e mobile
- ☐ Footer está consistente
- ☐ Sidebar sticky funciona em desktop
- ☐ Mobile adapta layout corretamente
- ☐ Estados loading existem
- ☐ Estados empty existem
- ☐ Estados error existem
- ☐ Disabled states são claros
- ☐ Toasts aparecem corretamente

- ☐ Textos estão legíveis
- ☐ Contraste está adequado
- ☐ Navegação está clara
- ☐ Interações têm feedback visual
- ☐ Animações são suaves e discretas

QA técnico

- ☐ TypeScript strict sem erros graves
- ☐ ESLint sem erros críticos
- ☐ Serviços desacoplados da UI
- ☐ Stores separadas por contexto
- ☐ Validações com RHF + Zod
- ☐ Estado persiste em localStorage quando necessário
- ☐ Mocks carregam corretamente
- ☐ Projeto inicia sem quebra
- ☐ Rotas navegam sem erro
- ☐ README existe e está completo

Finalidade desta seção

Esta versão separada do checklist de QA facilita:

- revisão manual do projeto;
- homologação interna por etapa;
- acompanhamento de entrega no Energent;
- validação rápida por blocos funcional, visual e técnico.

Prompt Mestre — Versão 4

Versão 4 separada para execução no Energent

AquaVista Tickets • pacote documental

Documento separado para colagem direta no Energent. Mantém o escopo executivo, a stack obrigatória e a ordem de execução por fases.

Conteúdo do prompt



Você é um engenheiro principal full-stack responsável por construir uma aplicação web original de venda de ingressos para parque aquático, com arquitetura moderna, regras de negócio realistas, UX premium e navegação completa.

IMPORTANTE

- Este projeto deve ser **ORIGINAL**.
- Não copie marca, nome, logo, textos, imagens, layout exato, HTML, CSS, assets, identidade visual, composição, rotas proprietárias ou estrutura literal de qualquer site real.
- O objetivo é criar uma solução própria com equivalência funcional, profundidade comercial semelhante e ótima qualidade técnica.

OBJETIVO

Construir um protótipo premium de ticketing para parque aquático com:

- seleção de data
- preços dinâmicos por data
- produtos simples
- passaportes de múltiplos dias
- combos multiparque
- ingressos especiais
- cabanas
- minicart
- carrinho
- checkout em múltiplas etapas
- login/cadastro
- área do cliente
- FAQ
- política comercial
- conteúdo institucional

- regras visíveis de elegibilidade, disponibilidade, estoque e pagamento

MARCA FICTÍCIA

- Brand: AquaVista Tickets
- Main park: AquaVista Water Park
- Secondary park: Bosque Aventura
- Premium area: AquaVista Cabanas

STACK OBRIGATÓRIA

- Next.js 14+ App Router
- React 18+
- TypeScript strict
- Tailwind CSS
- shadcn/ui
- Zustand
- TanStack Query
- React Hook Form
- Zod
- date-fns
- Framer Motion
- Lucide React
- Sonner
- MSW ou mock service interno
- Vitest
- Testing Library
- Playwright
- ESLint
- Prettier

ARQUITETURA

Crie estrutura organizada com:

- app routes
- components reutilizáveis
- services
- stores
- mocks
- validators
- tests
- README

ROTAS

- /
- /tickets
- /water-park/p
- /water-park/p?date=YYYY-MM-DD
- /cart
- /checkout/identify
- /checkout/customer
- /checkout/payment
- /checkout/review
- /checkout/success
- /account/login
- /account/register
- /account/orders
- /account/orders/[id]
- /faq
- /purchase-policy

REGRAS DE NEGÓCIO

- Data obrigatória para cálculo de preço final

- Datas passadas bloqueadas
- Datas esgotadas indisponíveis
- Preço varia por data
- Pix pode exibir preço preferencial
- Compra para uso no mesmo dia restringe pagamento para Pix
- Passaporte 2 dias exige 2 datas
- Passaporte 10 dias exige 10 datas
- Combo exige datas dos dois parques
- Cabana exige ingresso principal no carrinho
- Limite máximo por pedido
- Itens especiais exibem elegibilidade
- Carrinho vazio não avança
- Checkout exige aceite da política
- Carrinho e resumo recalculam automaticamente
- Cupom válido recalcula total
- Cupom inválido mostra erro
- Finalização gera pedido mock

PÁGINAS PRINCIPAIS

1. Landing institucional
2. Hub de ingressos
3. Página principal do parque com calendário + catálogo + sidebar sticky
4. Carrinho
5. Checkout 5 etapas
6. Conta do cliente
7. FAQ
8. Política comercial

COMPONENTES OBRIGATÓRIOS

- Header
- Footer
- Top promo bar
- Calendar
- Product cards
- Quantity stepper
- Sticky trip summary
- Mini cart drawer
- Checkout stepper
- Identify form
- Customer form
- Payment selector
- Review panel
- Success panel
- Login form
- Register form
- Orders list
- FAQ accordion
- Policy sections
- Skeletons
- Empty state
- Error state
- Toasts

TESTES

Criar:

- testes unitários
- testes de integração
- testes E2E com Playwright

ENTREGÁVEL

Entregue a implementação completa em código, com arquivos reais, rotas funcionais, dados mock, testes e README.

ORDEM DE EXECUÇÃO

Execute em fases:

1. estrutura + types + mocks
2. layout + páginas públicas
3. catálogo + calendário + pricing + minicart
4. carrinho + checkout
5. conta + pedidos + FAQ + política
6. testes + refinamento final + README

Ao final de cada fase:

- liste os arquivos criados
- resuma o que foi implementado
- indique critérios de aceite atendidos
- só então avance

Finalidade desta seção

Esta versão separada do prompt mestre facilita:

- colagem direta no Energent;
- preservação da instrução original da Versão 4;
- execução por fases com menor ambiguidade;
- uso como artefato operacional independente do restante do pacote documental.

Prompts por Fase — Versão 4

Versão 4 separada para execução no Energent

AquaVista Tickets • pacote documental

Cada fase foi isolada para facilitar execução incremental, validação e retomada do projeto.

FASE 1 — Estrutura, tipos, mocks e base do projeto

Implemente a Fase 1 do projeto AquaVista Tickets.

OBJETIVO DA FASE

Criar a base do projeto com stack, estrutura, tipos, mocks e serviços iniciais.

TAREFAS

1. Inicializar projeto com:

- Next.js 14+
- React 18+
- TypeScript strict
- Tailwind
- shadcn/ui
- Zustand
- TanStack Query
- React Hook Form
- Zod
- Framer Motion
- date-fns
- Sonner
- Vitest
- Testing Library
- Playwright
- ESLint
- Prettier

2. Criar estrutura de pastas:

- `/app`
- `/components`
- `/lib`
- `/services`

- `/store`
- `/mocks`
- `/tests`
- `/public`

3. Criar tipos de domínio:

- `Park`
- `VisitDate`
- `ProductCategory`
- `EligibilityRule`
- `TicketProduct`
- `CartItem`
- `CustomerAddress`
- `Customer`
- `Order`
- `Coupon`
- `PaymentMethod`
- `PaymentStatus`
- `CheckoutStep`

4. Criar seed mock inicial:

- `parks.json`
- `categories.json`
- `products.json`
- `availability.json`
- `coupons.json`
- `users.json`
- `faq.json`
- `policy.json`

5. Criar serviços base:

- `catalogService`
- `availabilityService`
- `pricingService`
- `authService`
- `cartService`
- `checkoutService`
- `ordersService`
- `policyService`

6. Criar stores Zustand:

- `useCatalogStore`
- `useCartStore`
- `useCheckoutStore`
- `useSessionStore`

7. Criar arquivo de constantes e helpers:

- route constants
- date helpers
- currency formatter
- validation helpers

RESULTADO ESPERADO

- Projeto compila
- Estrutura pronta
- Mocks carregáveis
- Types consistentes
- Serviços e stores iniciais criados

AO FINAL

- mostrar árvore de arquivos criada
- mostrar resumo técnico da fase
- apontar pendências da Fase 2

FASE 2 — Layout global e páginas públicas

Implemente a Fase 2 do projeto AquaVista Tickets.

OBJETIVO DA FASE

Construir layout global e páginas públicas iniciais com identidade original.

TAREFAS

1. Criar layout global:

- `AppHeader`
- `TopPromoBar`
- `MainNavigation`
- `MobileMenu`
- `AppFooter`
- `FooterPayments`
- `FooterSecurity`

2. Criar design system base:

- tokens de cores
- radius
- spacing
- sombras
- tipografia

3. Implementar páginas:

- `/`
- `/tickets`
- `/faq`
- `/purchase-policy`

4. Página `/`

- hero institucional original
- destaque de benefícios
- cards promocionais

- CTA para `/tickets`
 - teaser FAQ
 - CTA final
5. Página `/tickets`
- hub de compra
 - cards dos parques
 - CTA para `/water-park/p`
6. Página `/faq`
- busca
 - tabs/categorias
 - accordions
7. Página `/purchase-policy`
- navegação por seções
 - conteúdo original da política
8. Estados globais:
- loading
 - empty
 - error
 - toasts

RESULTADO ESPERADO

- páginas públicas navegáveis
- visual premium e original
- conteúdo institucional próprio

AO FINAL

- listar arquivos criados
- confirmar rotas funcionais
- indicar o que falta para a Fase 3

FASE 3 — Página principal do parque, calendário, catálogo, preços e minicart

Implemente a Fase 3 do projeto AquaVista Tickets.

OBJETIVO DA FASE

Construir a experiência principal de compra na rota `/water-park/p`.

TAREFAS

1. Implementar rota:

- `/water-park/p`
- suportar query param `?date=YYYY-MM-DD`

2. Criar componentes:

- `VisitCalendar`
- `CalendarDayCell`
- `CalendarLegend`
- `CalendarTooltip`
- `CalendarMonthControls`
- `SelectedDatesPanel`
- `ProductCategorySection`
- `ProductCard`
- `ProductBenefits`
- `ProductRules`
- `ProductPriceBlock`
- `ProductExpandDetails`
- `QuantityStepper`
- `SoldOutState`
- `EligibilityAlert`
- `ComboDateSelector`
- `PassportDateSelector`
- `CabanaInfoBadge`

- `StickyTripSummary`
- `MiniCartDrawer`

3. Implementar lógica:

- calendário com dois meses
- legenda visual por faixa de preço
- datas passadas bloqueadas
- datas sold out bloqueadas
- tooltip de preço
- mudança visual ao selecionar data
- mudança de preço nos cards ao selecionar data
- regras de disponibilidade por produto
- cards agrupados por categoria

4. Seções obrigatórias:

- Ingressos para 1 dia
- Passaportes
- Combos
- Ingressos especiais
- Cabanas

5. Implementar add to cart com validação:

- ingresso simples
- passaporte
- combo
- especial
- cabana

6. Implementar minicart:

- drawer
- resumo dos itens
- subtotal

- editar/remover
- CTA para carrinho

7. Implementar UX:

- skeleton loading
- hover
- animações com Framer Motion
- expandir "Saiba mais"
- sidebar sticky

RESULTADO ESPERADO

- `/water-park/p` funcional e convincente
- calendário integrado ao catálogo
- minicart funcionando
- regras visíveis refletidas no front

AO FINAL

- listar arquivos criados
- mostrar regras implementadas
- confirmar prontidão para Fase 4

FASE 4 — Carrinho e checkout completo

Implemente a Fase 4 do projeto AquaVista Tickets.

OBJETIVO DA FASE

Construir carrinho e checkout completo em 5 etapas.

TAREFAS

1. Implementar rotas:

- `/cart`
- `/checkout/identify`
- `/checkout/customer`
- `/checkout/payment`

- `/checkout/review`
- `/checkout/success`

2. Componentes:

- `CartLineItem`
- `CartSummaryCard`
- `CouponInput`
- `CartEmptyState`
- `CheckoutStepper`
- `IdentifyStepForm`
- `CustomerStepForm`
- `PaymentStepSelector`
- `ReviewStepPanel`
- `PolicyCheckbox`
- `OrderSuccessPanel`

3. `/cart`

- listar itens
- editar quantidade
- remover
- aplicar cupom
- recalcular subtotal, desconto e total
- validar carrinho vazio
- CTA continuar

4. `/checkout/identify`

- capturar e-mail
- sugerir conta existente
- login mock
- opção social mock

5. `/checkout/customer`

- formulário completo com RHF + Zod
- validar campos obrigatórios
- resumo lateral

6. `/checkout/payment`

- Pix
- cartão
- wallet
- restringir métodos quando a visita for hoje

7. `/checkout/review`

- revisar cliente
- revisar itens
- revisar datas
- revisar pagamento
- exigir aceite da política
- finalizar pedido

8. `/checkout/success`

- número do pedido
- status
- instruções
- CTA para pedidos
- CTA para início

9. Regras:

- carrinho vazio não avança
- pagamento do mesmo dia só Pix
- cabana sem ingresso falha antes
- política obrigatória
- pedido mock persistido

RESULTADO ESPERADO

- checkout completo e navegável
- validações funcionando
- pedido mock criado com sucesso

AO FINAL

- listar arquivos criados
- indicar fluxos cobertos
- confirmar prontidão para Fase 5

FASE 5 — Conta, pedidos, FAQ refinado e política final

Implemente a Fase 5 do projeto AquaVista Tickets.

OBJETIVO DA FASE

Concluir experiência pós-compra e conteúdo institucional final.

TAREFAS

1. Implementar rotas:

- `/account/login`
- `/account/register`
- `/account/orders`
- `/account/orders/[id]`

2. Componentes:

- `LoginForm`
- `RegisterForm`
- `OrdersList`
- `OrderDetailView`

3. `/account/login`

- login mock
- recuperação mock
- link para cadastro

4. `/account/register`

- cadastro mock
 - validação
5. `/account/orders`
- lista de pedidos
 - status
 - valor
 - data
 - link para detalhe
6. `/account/orders/[id]`
- itens
 - datas
 - forma de pagamento
 - status
 - instruções de uso
7. Refinar `/faq`
- categorias
 - busca
 - conteúdo completo
8. Refinar `/purchase-policy`
- documento completo
 - seções claras
 - texto original
9. Garantir persistência mock local:
- sessão
 - carrinho
 - pedidos

RESULTADO ESPERADO

- área do cliente funcional

- histórico de pedidos visível
- conteúdo institucional consolidado

AO FINAL

- listar arquivos criados
- mostrar rotas finalizadas
- confirmar prontidão para Fase 6

FASE 6 — Testes, refinamento e README

Implemente a Fase 6 do projeto AquaVista Tickets.

OBJETIVO DA FASE

Finalizar qualidade, testes e documentação.

TAREFAS

1. Criar testes unitários:

- `pricing.service`
- `availability.service`
- `cart.service`
- `checkout.service`
- `useCartStore`
- `useCheckoutStore`
- validators Zod

2. Criar testes de integração:

- adicionar ingresso simples
- adicionar passaporte
- adicionar combo
- validar cabana sem ingresso
- fluxo `identify -> customer -> payment -> review`

3. Criar testes E2E com Playwright:

- compra de ingresso 1 dia

- compra de passaporte 2 dias
- compra de combo
- cabana sem ingresso
- restrição de pagamento no mesmo dia

4. Refinar UX:

- skeletons
- disabled states
- mensagens inline
- banners
- acessibilidade
- foco
- labels
- feedback de erro

5. Criar README completo:

- visão geral
- stack
- setup
- execução
- testes
- estrutura
- decisões técnicas
- troca de mocks por API real

RESULTADO ESPERADO

- projeto estável
- testes cobrindo fluxos principais
- README pronto
- qualidade de demo executiva

AO FINAL

- listar arquivos finais
- indicar cobertura dos testes
- confirmar conclusão do projeto

Finalidade desta seção

Esta versão separada dos prompts por fase facilita:

- execução incremental no Energent;
- validação fase a fase;
- retomada do projeto sem perda de contexto;
- uso operacional como checklist de avanço por etapa.

Seed JSON Inicial — Versão 4

Versão 4 separada para execução no Energent

AquaVista Tickets • pacote documental

Base mock inicial para `parks`, `categories`, `products`, `availability`, `coupons`, `users`, `FAQ` e `policy`.

`parks.json`

```
[
  {
    "id": "park-av-water",
    "slug": "aqua-vista-water-park",
    "name": "AquaVista Water Park",
    "shortDescription": "Parque aquático premium com atrações para toda a família.",
    "heroImage": "/images/parks/aquavista-water.jpg",
    "benefits": [
      "Atrações para todas as idades",
      "Compra online com praticidade",
      "Experiência premium e segura"
    ]
  },
  {
    "id": "park-bosque-aventura",
```

```

    "slug": "bosque-aventura",
    "name": "Bosque Aventura",
    "shortDescription": "Parque complementar com experiênci
as ao ar livre e aventura.",
    "heroImage": "/images/parks/bosque-aventura.jpg",
    "benefits": [
      "Experiência em contato com a natureza",
      "Ideal para complementar a viagem",
      "Diversão ao ar livre"
    ]
  }
]

```

Observação do arquivo

Este conteúdo foi mantido como base do arquivo `parks.json` no pacote do projeto e no bloco de seed JSON inicial para uso operacional no Energent.

`categories.json`

```

[
  {
    "id": "cat-single-day",
    "slug": "single-day",
    "title": "Ingressos para 1 dia",
    "description": "Escolha a melhor data para sua visit
a.",
    "displayOrder": 1
  },
  {
    "id": "cat-passports",
    "slug": "passports",
    "title": "Passaportes",
    "description": "Mais dias para aproveitar com flexibili
dade.",
    "displayOrder": 2
  },
  {
    "id": "cat-combos",

```

```

    "slug": "combos",
    "title": "Combos",
    "description": "Combine experiências entre parques.",
    "displayOrder": 3
  },
  {
    "id": "cat-special",
    "slug": "special-tickets",
    "title": "Ingressos especiais",
    "description": "Condições específicas para públicos ele
gíveis.",
    "displayOrder": 4
  },
  {
    "id": "cat-cabanas",
    "slug": "cabanas",
    "title": "Cabanas",
    "description": "Conforto premium com reserva dedicad
a.",
    "displayOrder": 5
  }
]

```

Observação do arquivo

Este conteúdo foi mantido como base do arquivo `categories.json` no pacote do projeto e no bloco de seed JSON inicial para uso operacional no Energent.

`products.json`

```

[
  {
    "id": "prod-adult-day",
    "slug": "adult-day-pass",
    "parkId": "park-av-water",
    "categoryId": "cat-single-day",
    "type": "single_day",
    "audience": "adult",
    "title": "Adult Day Pass",

```

```

    "shortDescription": "Acesso de 1 dia ao parque aquático.",
    "longDescription": "Ingresso individual com acesso às atrações disponíveis na data escolhida.",
    "benefits": [
        "Acesso de 1 dia",
        "Compra online rápida",
        "Entrada vinculada à data selecionada"
    ],
    "rules": [
        "Válido apenas para a data escolhida",
        "Sujeito à disponibilidade",
        "Limite por pedido aplicado"
    ],
    "basePrice": 289,
    "pixPrice": 269,
    "creditPrice": 289,
    "requiresDateSelection": true,
    "requiredDatesCount": 1,
    "linkedParkIds": [],
    "eligibilityRules": [],
    "maxPerOrder": 10,
    "stockMode": "by_date",
    "active": true,
    "tags": [
        "popular"
    ]
},
{
    "id": "prod-child-day",
    "slug": "child-day-pass",
    "parkId": "park-av-water",
    "categoryId": "cat-single-day",
    "type": "single_day",
    "audience": "child",
    "title": "Child Day Pass",
    "shortDescription": "Acesso infantil de 1 dia.",
    "longDescription": "Ingresso infantil com acesso ao par

```

```

que na data escolhida.",
  "benefits": [
    "Acesso infantil de 1 dia",
    "Compra antecipada online",
    "Vinculado à data selecionada"
  ],
  "rules": [
    "Sujeito à política de faixa etária",
    "Válido apenas para a data escolhida"
  ],
  "basePrice": 259,
  "pixPrice": 239,
  "creditPrice": 259,
  "requiresDateSelection": true,
  "requiredDatesCount": 1,
  "linkedParkIds": [],
  "eligibilityRules": [],
  "maxPerOrder": 10,
  "stockMode": "by_date",
  "active": true,
  "tags": []
},
{
  "id": "prod-flex-day",
  "slug": "flex-day-promo",
  "parkId": "park-av-water",
  "categoryId": "cat-single-day",
  "type": "single_day",
  "audience": "mixed",
  "title": "Flex Day Promo",
  "shortDescription": "Opção promocional em datas selecionadas.",
  "longDescription": "Produto promocional com disponibilidade limitada por data.",
  "benefits": [
    "Melhor preço em datas elegíveis",
    "Boa opção para compra antecipada"
  ],

```



```

    "rules": [
      "Disponível apenas em datas promocionais",
      "Pode esgotar rapidamente"
    ],
    "basePrice": 249,
    "pixPrice": 229,
    "creditPrice": 249,
    "requiresDateSelection": true,
    "requiredDatesCount": 1,
    "linkedParkIds": [],
    "eligibilityRules": [],
    "maxPerOrder": 6,
    "stockMode": "by_date",
    "active": true,
    "tags": [
      "promo"
    ]
  },
  {
    "id": "prod-passport-2",
    "slug": "passport-2-days",
    "parkId": "park-av-water",
    "categoryId": "cat-passports",
    "type": "passport",
    "audience": "mixed",
    "title": "Passport 2 Days",
    "shortDescription": "Aproveite 2 dias no parque.",
    "longDescription": "Passaporte com seleção obrigatória
de 2 datas válidas.",
    "benefits": [
      "2 dias de acesso",
      "Maior flexibilidade",
      "Custo-benefício superior"
    ],
    "rules": [
      "Exige exatamente 2 datas",
      "Datas sujeitas à disponibilidade"
    ],

```

```

    "basePrice": 399,
    "pixPrice": 379,
    "creditPrice": 399,
    "requiresDateSelection": true,
    "requiredDatesCount": 2,
    "linkedParkIds": [],
    "eligibilityRules": [],
    "maxPerOrder": 6,
    "stockMode": "by_date",
    "active": true,
    "tags": [
      "best-seller"
    ]
  },
  {
    "id": "prod-passport-10",
    "slug": "passport-10-days",
    "parkId": "park-av-water",
    "categoryId": "cat-passports",
    "type": "passport",
    "audience": "mixed",
    "title": "Passport 10 Days",
    "shortDescription": "Mais tempo para aproveitar em vári
as visitas.",
    "longDescription": "Passaporte premium com seleção de 1
0 datas válidas.",
    "benefits": [
      "10 acessos",
      "Ideal para estadias longas",
      "Melhor custo por visita"
    ],
    "rules": [
      "Exige 10 datas",
      "Sujeito à janela de validade"
    ],
    "basePrice": 659,
    "pixPrice": 629,
    "creditPrice": 659,

```

```

    "requiresDateSelection": true,
    "requiredDatesCount": 10,
    "linkedParkIds": [],
    "eligibilityRules": [],
    "maxPerOrder": 4,
    "stockMode": "by_date",
    "active": true,
    "tags": [
      "premium"
    ]
  },
  {
    "id": "prod-combo-1",
    "slug": "combo-water-park-adventure-park",
    "parkId": "park-av-water",
    "categoryId": "cat-combos",
    "type": "combo",
    "audience": "mixed",
    "title": "Combo Water Park + Adventure Park",
    "shortDescription": "Combine os dois parques em uma só compra.",
    "longDescription": "Combo que exige seleção de data para o parque aquático e para o parque complementar.",
    "benefits": [
      "Experiência completa",
      "Compra combinada",
      "Melhor organização da viagem"
    ],
    "rules": [
      "Exige datas para dois parques",
      "Sujeito à disponibilidade em ambos"
    ],
    "basePrice": 559,
    "pixPrice": 529,
    "creditPrice": 559,
    "requiresDateSelection": true,
    "requiredDatesCount": 2,
    "linkedParkIds": [

```

```

        "park-bosque-aventura"
    ],
    "eligibilityRules": [],
    "maxPerOrder": 6,
    "stockMode": "by_date",
    "active": true,
    "tags": [
        "combo"
    ]
},
{
    "id": "prod-combo-2",
    "slug": "combo-multi-day",
    "parkId": "park-av-water",
    "categoryId": "cat-combos",
    "type": "combo",
    "audience": "mixed",
    "title": "Combo Multi Day",
    "shortDescription": "Mais de uma experiência em uma compra unificada.",
    "longDescription": "Combo com lógica estendida de datas para uma jornada multiparque.",
    "benefits": [
        "Mais flexibilidade",
        "Combinação de experiências"
    ],
    "rules": [
        "Exige múltiplas datas",
        "Disponibilidade pode variar por parque"
    ],
    "basePrice": 799,
    "pixPrice": 759,
    "creditPrice": 799,
    "requiresDateSelection": true,
    "requiredDatesCount": 3,
    "linkedParkIds": [
        "park-bosque-aventura"
    ],

```

```

    "eligibilityRules": [],
    "maxPerOrder": 4,
    "stockMode": "by_date",
    "active": true,
    "tags": [
      "combo",
      "multi"
    ]
  },
  {
    "id": "prod-senior",
    "slug": "senior-access",
    "parkId": "park-av-water",
    "categoryId": "cat-special",
    "type": "special",
    "audience": "senior",
    "title": "Senior Access",
    "shortDescription": "Condição especial para público ele
gível.",
    "longDescription": "Ingresso especial sujeito às regras
de elegibilidade e comprovação.",
    "benefits": [
      "Condição especial",
      "Compra online prática"
    ],
    "rules": [
      "Elegibilidade sujeita à validação",
      "Documento comprobatório pode ser exigido"
    ],
    "basePrice": 169,
    "pixPrice": 159,
    "creditPrice": 169,
    "requiresDateSelection": true,
    "requiredDatesCount": 1,
    "linkedParkIds": [],
    "eligibilityRules": [
      {
        "code": "senior-age",

```

```

        "label": "Elegibilidade por idade",
        "description": "Sujeito às regras de elegibilidade
do produto."
    }
],
    "maxPerOrder": 4,
    "stockMode": "by_date",
    "active": true,
    "tags": [
        "special"
    ]
},
{
    "id": "prod-pcd",
    "slug": "pcd-access",
    "parkId": "park-av-water",
    "categoryId": "cat-special",
    "type": "special",
    "audience": "pcd",
    "title": "PCD Access",
    "shortDescription": "Ingresso especial para público ele
gível.",
    "longDescription": "Produto especial com regras própria
s de elegibilidade.",
    "benefits": [
        "Compra antecipada",
        "Condição especial"
    ],
    "rules": [
        "Sujeito à elegibilidade",
        "Documento comprobatório pode ser solicitado"
    ],
    "basePrice": 169,
    "pixPrice": 159,
    "creditPrice": 169,
    "requiresDateSelection": true,
    "requiredDatesCount": 1,
    "linkedParkIds": [],

```

```

    "eligibilityRules": [
      {
        "code": "pcd-proof",
        "label": "Comprovação de elegibilidade",
        "description": "Sujeito às condições específicas do
ingresso."
      }
    ],
    "maxPerOrder": 4,
    "stockMode": "by_date",
    "active": true,
    "tags": [
      "special"
    ]
  },
  {
    "id": "prod-pregnant",
    "slug": "pregnant-access",
    "parkId": "park-av-water",
    "categoryId": "cat-special",
    "type": "special",
    "audience": "pregnant",
    "title": "Pregnant Access",
    "shortDescription": "Ingresso especial com condição esp
ecífica.",
    "longDescription": "Ingresso especial para público eleg
ível, com observações importantes de uso.",
    "benefits": [
      "Compra online",
      "Condição especial"
    ],
    "rules": [
      "Sujeito à elegibilidade",
      "Pode exigir confirmação das condições aplicáveis"
    ],
    "basePrice": 169,
    "pixPrice": 159,
    "creditPrice": 169,
  }
}

```

```

    "requiresDateSelection": true,
    "requiredDatesCount": 1,
    "linkedParkIds": [],
    "eligibilityRules": [
      {
        "code": "pregnant-eligibility",
        "label": "Condição especial",
        "description": "Sujeito às condições específicas do
produto."
      }
    ],
    "maxPerOrder": 4,
    "stockMode": "by_date",
    "active": true,
    "tags": [
      "special"
    ]
  },
  {
    "id": "prod-cabana-areia",
    "slug": "cabana-areia",
    "parkId": "park-av-water",
    "categoryId": "cat-cabanas",
    "type": "cabana",
    "audience": "mixed",
    "title": "Cabana Areia",
    "shortDescription": "Espaço privativo para até 6 pessoa
s.",
    "longDescription": "Cabana premium sujeita à disponibil
idade por data.",
    "benefits": [
      "Espaço reservado",
      "Mais conforto durante a visita"
    ],
    "rules": [
      "Exige ingresso principal",
      "Disponibilidade limitada por data"
    ],

```



```

    "basePrice": 790,
    "pixPrice": 760,
    "creditPrice": 790,
    "requiresDateSelection": true,
    "requiredDatesCount": 1,
    "linkedParkIds": [],
    "eligibilityRules": [],
    "maxPerOrder": 1,
    "stockMode": "by_capacity",
    "capacity": 6,
    "active": true,
    "tags": [
      "premium",
      "cabana"
    ]
  },
  {
    "id": "prod-cabana-costa",
    "slug": "cabana-costa",
    "parkId": "park-av-water",
    "categoryId": "cat-cabanas",
    "type": "cabana",
    "audience": "mixed",
    "title": "Cabana Costa",
    "shortDescription": "Espaço premium para até 8 pessoas.",
    "longDescription": "Cabana de categoria intermediária com disponibilidade limitada.",
    "benefits": [
      "Mais espaço",
      "Conforto premium"
    ],
    "rules": [
      "Exige ingresso principal",
      "Reserva sujeita à disponibilidade"
    ],
    "basePrice": 990,
    "pixPrice": 950,

```

```

    "creditPrice": 990,
    "requiresDateSelection": true,
    "requiredDatesCount": 1,
    "linkedParkIds": [],
    "eligibilityRules": [],
    "maxPerOrder": 1,
    "stockMode": "by_capacity",
    "capacity": 8,
    "active": true,
    "tags": [
      "premium",
      "cabana"
    ]
  },
  {
    "id": "prod-cabana-ilha",
    "slug": "cabana-ilha",
    "parkId": "park-av-water",
    "categoryId": "cat-cabanas",
    "type": "cabana",
    "audience": "mixed",
    "title": "Cabana Ilha",
    "shortDescription": "Cabana ampla para grupos maiores.",
    "longDescription": "Cabana premium de maior capacidade para grupos e famílias.",
    "benefits": [
      "Maior capacidade",
      "Experiência premium ampliada"
    ],
    "rules": [
      "Exige ingresso principal",
      "Reserva sujeita à disponibilidade"
    ],
    "basePrice": 1390,
    "pixPrice": 1340,
    "creditPrice": 1390,
    "requiresDateSelection": true,

```

```

    "requiredDatesCount": 1,
    "linkedParkIds": [],
    "eligibilityRules": [],
    "maxPerOrder": 1,
    "stockMode": "by_capacity",
    "capacity": 16,
    "active": true,
    "tags": [
      "premium",
      "cabana",
      "group"
    ]
  }
]

```

Observação do arquivo

Este conteúdo foi mantido como base do arquivo `products.json` no pacote do projeto e no bloco de seed JSON inicial para uso operacional no Energent.

`availability.json`

```

[
  {
    "date": "2026-04-10",
    "priceLevel": "lowest",
    "status": "available",
    "basePriceAdult": 269,
    "basePriceChild": 239,
    "pixDiscountPercent": 7,
    "tags": [
      "promo"
    ]
  },
  {
    "date": "2026-04-11",
    "priceLevel": "standard",
    "status": "available",
    "basePriceAdult": 289,

```

```

    "basePriceChild": 259,
    "pixDiscountPercent": 6,
    "tags": []
  },
  {
    "date": "2026-04-12",
    "priceLevel": "peak",
    "status": "available",
    "basePriceAdult": 319,
    "basePriceChild": 289,
    "pixDiscountPercent": 5,
    "tags": [
      "high-demand"
    ]
  },
  {
    "date": "2026-04-13",
    "priceLevel": "sold_out",
    "status": "sold_out",
    "basePriceAdult": 0,
    "basePriceChild": 0,
    "pixDiscountPercent": 0,
    "tags": [
      "sold-out"
    ]
  },
  {
    "date": "2026-04-14",
    "priceLevel": "standard",
    "status": "low_availability",
    "basePriceAdult": 294,
    "basePriceChild": 264,
    "pixDiscountPercent": 5,
    "tags": [
      "few-left"
    ]
  },
  {

```

```
    "date": "2026-05-01",
    "priceLevel": "lowest",
    "status": "available",
    "basePriceAdult": 265,
    "basePriceChild": 235,
    "pixDiscountPercent": 7,
    "tags": [
      "promo"
    ]
  },
  {
    "date": "2026-05-02",
    "priceLevel": "peak",
    "status": "available",
    "basePriceAdult": 325,
    "basePriceChild": 295,
    "pixDiscountPercent": 5,
    "tags": [
      "holiday"
    ]
  },
  {
    "date": "2026-05-03",
    "priceLevel": "sold_out",
    "status": "sold_out",
    "basePriceAdult": 0,
    "basePriceChild": 0,
    "pixDiscountPercent": 0,
    "tags": [
      "sold-out"
    ]
  }
]
```

Observação do arquivo

Este conteúdo foi mantido como base do arquivo `availability.json` no pacote do projeto e no bloco de seed JSON inicial para uso operacional no Energent.

`coupons.json`

```
[
  {
    "code": "AQUA10",
    "type": "percentage",
    "value": 10,
    "active": true
  },
  {
    "code": "VIP20",
    "type": "percentage",
    "value": 20,
    "active": true
  },
  {
    "code": "FAMILY50",
    "type": "fixed",
    "value": 50,
    "active": true
  }
]
```

Observação do arquivo

Este conteúdo foi mantido como base do arquivo `coupons.json` no pacote do projeto e no bloco de seed JSON inicial para uso operacional no Energent.

`users.json`

```
[
  {
    "id": "user-001",
    "firstName": "Marina",
    "lastName": "Souza",
    "email": "marina@example.com",
    "phone": "85999999999",
    "document": "12345678900",
    "orders": [
```

```

{
  "id": "order-001",
  "number": "AV-20260401-0001",
  "paymentMethod": "pix",
  "paymentStatus": "approved",
  "policyAccepted": true,
  "createdAt": "2026-04-01T10:30:00.000Z",
  "total": 538,
  "visitInstructions": [
    "Apresente seu comprovante no acesso.",
    "Verifique os dados da visita antes do uso."
  ],
  "items": [
    {
      "id": "item-001",
      "productId": "prod-adult-day",
      "productType": "single_day",
      "title": "Adult Day Pass",
      "quantity": 2,
      "selectedDates": [
        "2026-04-10"
      ],
      "selectedParks": [
        "park-av-water"
      ],
      "unitPrice": 269,
      "pixUnitPrice": 269,
      "subtotal": 538
    }
  ]
}
]

```

Observação do arquivo

Este conteúdo foi mantido como base do arquivo `users.json` no pacote do projeto e no bloco de seed JSON inicial para uso operacional no Energnet.

faq.json

```
[
  {
    "id": "faq-001",
    "category": "Compra",
    "question": "Como funciona a compra online?",
    "answer": "Você seleciona a data, escolhe os produtos, adiciona ao carrinho e conclui o checkout."
  },
  {
    "id": "faq-002",
    "category": "Compra",
    "question": "Posso comprar para outra pessoa?",
    "answer": "Sim, desde que os dados e condições aplicáveis sejam respeitados."
  },
  {
    "id": "faq-003",
    "category": "Datas",
    "question": "Os preços mudam conforme a data?",
    "answer": "Sim, os preços podem variar conforme disponibilidade e período."
  },
  {
    "id": "faq-004",
    "category": "Datas",
    "question": "Posso escolher datas futuras?",
    "answer": "Sim, desde que estejam disponíveis no calendário."
  },
  {
    "id": "faq-005",
    "category": "Passaportes",
    "question": "Como funciona o passaporte de 2 dias?",
    "answer": "Você deve selecionar exatamente duas datas válidas para utilizar o produto."
  },
]
```



```

{
  "id": "faq-006",
  "category": "Passaportes",
  "question": "O passaporte de 10 dias tem validade?",
  "answer": "Sim, o produto deve exibir claramente a regra de validade e janela de uso."
},
{
  "id": "faq-007",
  "category": "Combos",
  "question": "O combo exige duas datas?",
  "answer": "Sim, o sistema solicita a seleção das datas necessárias para cada parque incluído."
},
{
  "id": "faq-008",
  "category": "Combos",
  "question": "Os parques podem ter datas diferentes?",
  "answer": "Sim, conforme a configuração do produto e disponibilidade."
},
{
  "id": "faq-009",
  "category": "Especiais",
  "question": "Quem pode comprar ingressos especiais?",
  "answer": "Apenas públicos elegíveis conforme as regras descritas no produto."
},
{
  "id": "faq-010",
  "category": "Especiais",
  "question": "É necessário comprovar elegibilidade?",
  "answer": "Em alguns casos, sim. O produto informa essa condição."
},
{
  "id": "faq-011",
  "category": "Cabanas",

```

```

    "question": "Posso reservar cabana sem ingresso?",
    "answer": "Não. A reserva de cabana exige ao menos um ingresso principal no carrinho."
  },
  {
    "id": "faq-012",
    "category": "Cabanas",
    "question": "As cabanas têm limite de pessoas?",
    "answer": "Sim, cada cabana possui capacidade máxima própria."
  },
  {
    "id": "faq-013",
    "category": "Pagamento",
    "question": "Quais formas de pagamento estão disponíveis?",
    "answer": "O checkout pode oferecer Pix, cartão e carteira digital, conforme contexto da compra."
  },
  {
    "id": "faq-014",
    "category": "Pagamento",
    "question": "Por que algumas compras só aceitam Pix?",
    "answer": "Compras para uso no mesmo dia podem ter restrições operacionais de pagamento."
  },
  {
    "id": "faq-015",
    "category": "Pedidos",
    "question": "Onde vejo meus pedidos?",
    "answer": "Na área do cliente, em Meus Pedidos."
  },
  {
    "id": "faq-016",
    "category": "Pedidos",
    "question": "Receberei um número de pedido?",
    "answer": "Sim, ao finalizar a compra o sistema gera um identificador do pedido."
  }

```

```
    },
    {
      "id": "faq-017",
      "category": "Uso",
      "question": "Preciso apresentar algo no acesso?",
      "answer": "O pedido deve exibir instruções de uso e validação para entrada."
    },
    {
      "id": "faq-018",
      "category": "Uso",
      "question": "Posso alterar minha data depois?",
      "answer": "A política comercial deve informar as condições de remarcação."
    },
    {
      "id": "faq-019",
      "category": "Cancelamento",
      "question": "Como funciona o cancelamento?",
      "answer": "As condições devem estar descritas na política comercial do produto e da plataforma."
    },
    {
      "id": "faq-020",
      "category": "Atendimento",
      "question": "Onde encontro suporte?",
      "answer": "A área institucional deve apresentar canais de contato e horários de atendimento."
    }
  ]
```

Observação do arquivo

Este conteúdo foi mantido como base do arquivo `faq.json` no pacote do projeto e no bloco de seed JSON inicial para uso operacional no Energent.

`policy.json`

```
[
  {
    "id": "policy-001",
    "title": "Condições gerais de compra",
    "content": "A compra online está sujeita à disponibilidade, confirmação dos dados informados e regras específicas de cada produto."
  },
  {
    "id": "policy-002",
    "title": "Preços e disponibilidade",
    "content": "Os preços podem variar conforme a data, demanda e disponibilidade, sem garantia de manutenção após alteração de contexto de compra."
  },
  {
    "id": "policy-003",
    "title": "Uso do ingresso",
    "content": "Cada ingresso é válido conforme a descrição do produto, datas selecionadas e condições informadas durante a compra."
  },
  {
    "id": "policy-004",
    "title": "Passaportes",
    "content": "Passaportes exigem seleção e utilização conforme a quantidade de datas e validade previstas no produto."
  },
  {
    "id": "policy-005",
    "title": "Combos multiparque",
    "content": "Combos podem exigir a escolha de mais de uma data e estão sujeitos à disponibilidade de todos os parques envolvidos."
  },
  {
    "id": "policy-006",
```

```

    "title": "Ingressos especiais",
    "content": "Ingressos especiais são destinados exclusiv
amente a públicos elegíveis, conforme regras de cada produt
o."
  },
  {
    "id": "policy-007",
    "title": "Cabanas",
    "content": "A reserva de cabanas depende da compra de i
ngresso principal válido e está sujeita à disponibilidade e
capacidade da cabana escolhida."
  },
  {
    "id": "policy-008",
    "title": "Pagamento",
    "content": "Os meios de pagamento disponíveis podem var
iar conforme o contexto da compra, incluindo restrições ope
racionais aplicáveis."
  },
  {
    "id": "policy-009",
    "title": "Compras para o mesmo dia",
    "content": "Compras para uso no mesmo dia podem estar s
ujeitas a regras específicas de aprovação e meios de pagame
nto limitados."
  },
  {
    "id": "policy-010",
    "title": "Remarcação e cancelamento",
    "content": "As condições de remarcação e cancelamento d
evem seguir as regras aplicáveis ao produto e podem envolve
r diferença tarifária."
  },
  {
    "id": "policy-011",
    "title": "Atendimento",
    "content": "Os canais de suporte e orientações pós-comp
ra devem estar disponíveis nas páginas institucionais da pl

```

```
ataforma."  
  }  
]
```

Observação do arquivo

Este conteúdo foi mantido como base do arquivo `policy.json` no pacote do projeto e no bloco de seed JSON inicial para uso operacional no Energent.

Finalidade desta seção

Esta versão separada do seed inicial facilita:

- uso direto em execução no Energent;
- conferência do conteúdo mock por arquivo;
- desacoplamento entre documentação narrativa e base de dados inicial;
- preparação da pasta `seed_json/` com os arquivos individuais.

Critérios Playwright — Versão 4

Versão 4 separada para execução no Energent

AquaVista Tickets • pacote documental

Critérios de aceite dos cenários E2E, já quebrados por suíte para facilitar implementação e conferência.

Suíte 1 — single day purchase

- **Nome:** `buy-single-day.spec.ts`
- acessar `/water-park/p`
- selecionar data disponível
- adicionar 1 ingresso adulto
- abrir minicart
- ir para `/cart`
- avançar para `/checkout/identify`
- preencher e-mail
- avançar para `/checkout/customer`

- preencher dados válidos
- avançar para `/checkout/payment`
- selecionar Pix
- avançar para `/checkout/review`
- aceitar política
- finalizar pedido
- validar `/checkout/success`
- validar número do pedido visível

Suite 2 — passport 2 days

- **Nome:** `buy-passport-2-days.spec.ts`
- acessar `/water-park/p`
- selecionar 2 datas válidas
- adicionar passaporte 2 dias
- validar item no carrinho com 2 datas
- concluir checkout
- validar sucesso

Suite 3 — combo purchase

- **Nome:** `buy-combo.spec.ts`
- acessar `/water-park/p`
- selecionar data do parque principal
- selecionar data do parque complementar
- adicionar combo
- validar carrinho
- concluir checkout
- validar sucesso

Suite 4 — cabana restriction

- **Nome:** `cabana-requires-ticket.spec.ts`

- acessar `/water-park/p`
- tentar adicionar cabana sem ingresso principal
- validar mensagem de erro
- validar que item não foi adicionado

Suite 5 — same day payment restriction

- **Nome:** `same-day-payment-restriction.spec.ts`
- acessar `/water-park/p`
- selecionar a data de hoje
- adicionar ingresso
- seguir checkout
- chegar em `/checkout/payment`
- validar que apenas Pix está disponível
- validar cartão indisponível
- validar wallet indisponível

Suite 6 — invalid coupon

- **Nome:** `invalid-coupon.spec.ts`
- adicionar item ao carrinho
- inserir cupom inválido
- validar toast de erro
- validar total sem alteração

Suite 7 — account orders

- **Nome:** `account-orders.spec.ts`
- concluir uma compra mock
- acessar `/account/orders`
- validar pedido listado
- abrir `/account/orders/[id]`
- validar detalhes completos

Finalidade desta seção

Esta versão separada dos critérios de Playwright facilita:

- implementação das suítes E2E por arquivo;
- conferência dos cenários obrigatórios;
- uso como checklist de automação no Energent;
- validação rápida dos fluxos críticos do funil.

Índice de Entrega — Versão 4

Versão 4 separada para execução no Energent

AquaVista Tickets • pacote documental

Arquivos incluídos

- Documento principal consolidado da Versão 4.
- Complemento de mapas, rotas e catálogo de destinos.
- Seed adicional de destinos.
- Handoff técnico para Replit + Cursor AI.
- Índice/guia de entrega do pacote documental.

Pasta adicional

- `seed_json/` com os arquivos JSON separados para consumo operacional, testes, importação e reaproveitamento por ambiente.

Finalidade do índice de entrega

Este índice funciona como camada de organização do pacote final, facilitando:

- conferência dos artefatos entregues;
- separação entre documentação principal e arquivos de seed;
- repasse para execução em ambientes como Energent;
- validação rápida do que entra no pacote documental da Versão 4.

Observação operacional

Para uso em execução assistida, o ideal é manter esse índice como capa ou página-resumo do pacote, apontando para os documentos principais e para a

pasta `seed_json/`.

Recomendação final

Suba este documento no Notion no lugar do documento anterior. Ele está mais adequado para o Notion AI porque já traz não só o plano funcional, mas também a camada de modelagem de projeto, organização por databases e um manifesto mais detalhado de arquivos.

A melhor sequência depois da importação é:

1. Gerar databases.
2. Criar relações entre eles.
3. Transformar o manifesto em backlog executável por fase.

Fonte da consolidação

Baseado na Versão 4 consolidada, no complemento de mapas/rotas, no handoff operacional para Replit/Cursor e no índice de entrega documental, que juntos cobrem a base funcional, seeds, QA, arquitetura, baseline técnica de implementação e empacotamento para execução.